

BACHELOR THESIS
COMPUTER SCIENCE

HOW TO MAKE A
BICYCLE (NOT) FALL -
IMPLEMENTING A
STEERING CONTROL
FOR A
SELF-BALANCING
BICYCLE

ANNA RANZ

INSTITUTE OF COMPUTER SCIENCE
FACULTY OF COMPUTER SCIENCE AND MATHEMATICS
JOHANN WOLFGANG GOETHE-UNIVERSITY

SUPERVISED BY
PROF. DR.-ING. LARS HEDRICH

AUGUST 15, 2021

STATUTORY DECLARATION

I, Anna Ranz, herewith declare that I have composed the present thesis myself and without use of any other than the cited sources and aids.

ERKLÄRUNG ZUR ABSCHLUSSARBEIT (SCANNED VERSION IN APPENDIX E)

gemäß §25, Abs. 11 der Ordnung für den Bachelorstudien-
gang Informatik vom 06.Dezember 2010:

Hiermit versichere ich, Anna Ranz, dass ich die vor-
liegende Arbeit selbstständig und nur unter Verwendung
der angegebenen Hilfsmittel angefertigt habe.

Frankfurt am Main, August 15, 2021

A handwritten signature in black ink, appearing to read 'ARanz', written in a cursive style.

Anna Ranz

ZUSAMMENFASSUNG

Fahrräder sind nicht nur unsere täglichen Begleiter, sondern faszinieren auch als Subjekt der Modellierung und Regelung.

Die vorliegende Arbeit möchte eine kurze Einleitung in die Fahrradmodellierung und selbst-balancierende Fahrräder geben. Weiter dokumentiert sie den Entwurf und die Implementierung eines selbst-balancierenden Fahrradprototyps mit Lenkregelung.

ABSTRACT

Besides being a trusted sidekick in our daily lives bicycles daunt us as a problem of modeling and control theory. This work aims to give a short introduction into bicycle-modeling and self-balancing bicycles.

It further documents the implementation and design of a self-balancing bicycle prototype which uses steering-control.

AIM OF PROJECT

The present work is dedicated to laying the groundwork of designing and implementing a steering-control for a bicycle. The basis for the implementation are preexisting models of bicycles and corresponding control systems.

The work strives to summarize the theoretical background and to document the implementation of the self-balancing bicycle prototype with its successes and drawbacks in hope that this prototype may form a base for further exploration and experimentation by other students.

CONTENTS

Contents	vii
List of Figures	viii
1 Introduction	1
2 Background	3
2.1 What parameters has a bicycle?	3
2.2 A very short history of Bicycle Modeling and Self-Balancing Bicycles	7
2.3 Measuring Position	10
2.4 What is a DC motor and its motor constants?	11
2.5 Communication between Components	13
2.6 Controlling the System	14
2.7 And how can we model the prototype?	18
3 System Development	22
3.1 The Bike	23
3.2 The Mechanics of the Steering-Control	24
3.3 The Electronics of the Steering-Control	26
3.4 The Control Loop	30
3.5 The Implementation in Software	34
3.6 Measuring and Testing	34
4 Conclusion and Future Work	35
4.1 Modeling pain and future models	35
4.2 Wobbly screws and unreliable wiring	36
4.3 A RT-Linux without any RT and mysterious calibration outages	36
4.4 Finally	37
A Details of the first model	38
B Details of the Software Implementation	42
C Wiring Diagram	43
D List of Parts and used Connector Types	44
E Erklärung zur Abschlussarbeit	46
Glossary	47
Acronyms	48
Bibliography	49

LIST OF FIGURES

2.1	A urban racer versus a dutch bike with emphasis on their head angles	4
2.2	The world frame	5
2.3	a) yaw angle ψ seen from above b) roll angle ϕ seen from the front	5
2.4	Bicycle Parameters	6
2.5	Summary of advantages and disadvantages according to [Den+18]	9
2.6	EMG49 Motor Characteristic[Lim21a]	12
2.7	EMG49 Motor Characteristic[Lim21a]	12
2.8	a) Open loop system b) Closed loop system	15
2.9	Block diagram of a) flashlight and b) heater	16
2.10	Heater System Detail	16
2.11	Interconnections of linear systems. Series (a), parallel (b), and feedback (c) connections are shown [ÅM21, p.9-17] . . .	17
2.12	Tilt of the frame modeled as inverted pendulum	18
2.13	Single track model of the bicycle with the center of gravity as reference point	21
3.1	The complete platform being tested	22
3.2	Steering Control System Diagram	22
3.3	Static parameters of the chosen bicycle	23
3.4	The supply voltages	24
3.5	The gears of the first approach	25
3.6	The final version uses a toothed belt and two gears to steer	25
3.7	Gear transmission of steering motor to handlebar steering angle	26
3.8	Steering Control Hardware	26
3.9	Adafruit 9-DOF Absolute Orientation IMU Fusion Break-out[Ind21]	28
3.10	Remapping scheme P0 for the BNO055 chip [Sen20, p. 27]	28
3.11	Orientation of roll angle after axes remapping	28
3.12	EMG49 Properties[Lim21a], entries in red were approximated from the vendor-supplied information	29
3.13	Variables of motor control	30
3.14	The Control System Block Diagrams a) in the time domain b) in the frequency domain	31
3.15	The Motor Control Block Diagram	32
A.1	The Control System Block Diagram of the first modeling attempt	39
C.1	Wiring Diagram generated with fritzing[fri21]. The fritzing file is included in the accompanying source folder.	43

INTRODUCTION

*Beim Menschen ist es wie beim Velo. Nur wenn er fährt,
kann er bequem Balance halten.*

— ALBERT EINSTEIN [ISA07]

Since Karl von Drais invented the draisine between 1812 and 1817[WSP20, p. 6 ff] the bicycle in all its forms has been a constant companion of us humans. We use it for transportation of ourselves and wares, for sport and fun and in this work for math and control systems theory.

The importance of bicycles rises as the need for climate friendly transportation grows even more urgent. In times of a global climate crisis getting people to cycle instead of driving by car offers one of the easiest ways to reduce carbon emissions. The European non-profit European Cyclists' Federation estimated in 2018 that "cycling saves emissions equaling more than 16 million tons of CO₂ equivalents per year in the EU. This corresponds to the total yearly CO₂ emissions of a whole country like Croatia." [Fed18, p. 7]

At the same time we need to deal with a European population that is getting older. As Vera Bultink neatly summarizes in her dissertation on improving cycling safety for older cyclists: If we want more people cycling and our populace is getting older we need more research into bicycle stability and assisted cycling[Bul17]

Bicycles are an object of active research regarding their modelling and control and have been for more than 100 years. "The first substantial contribution to the theoretical bicycle literature was Whipple's seminal 1899 paper" [Lim06, p. 38]. Whipple's bicycle model consists of four rigid bodies. The rear frame with or without a rider, the front frame including the fork and handlebar and two wheels.

One might think the mathematics of such a model quite easy and the topic of bicycle dynamics a thoroughly researched topic - but that is not the case. As Wilson puts it quite nicely: "Unfortunately, the mathematics purporting to describe bicycle motion and self-stability is neither complete nor easily understandable." [WSP20, p. 350]

An impressive way to rebut theories on how and why bicycles self-stabilize is by building *Unrideable Bicycles*[ÅKL05]. These still self-stabilize and the search for the underlying causes of self-stability one so that one may say that there is a lot to be researched in the field of bicycle dynamics.

This work starts by presenting the most important bicycle parameters and their influence on the bike's ride characteristics. After the parameters are defined, a selection of previous works and their modeling techniques and modes of achieving self-balancing are presented.

To understand the underlying principles of the prototype, this work will touch on the theoretical background of control systems and introduces technologies used in the prototype build. It presents the

INTRODUCTION

chosen mechanical and electronic components and illustrates their strengths and drawbacks.

A model of the relationship between steering angle is derived and used in a fully documented controller design. At last software implementation of the controller, including the communication with the sensor and the motor, is discussed before concluding and discussing further work.

BACKGROUND

This chapter aims to give an overview of the underlying principles and techniques of modeling and controlling a bicycle.

A *bicycle* in this context may be defined as a human- or motor-powered, single-track vehicle which has two wheels attached to a frame, one behind the other.

This chapter starts with the question of how a bicycle can be defined and what parameters it possesses in section 2.1. With the basics defined the next section 2.2 shows a selection of bicycle models and bicycle balancing implementations.

The means of measuring position are discussed in section 2.3 and the basics of a DC-motor and its constants are described in section 2.4.

How the different components communicate is touched on in section 2.5 before introducing control systems in section 2.6.

In the last section 2.7 all the above is combined into a model for the prototype of a self-balancing bicycle which will be used for its implementation in the following chapter 3.

2.1 WHAT PARAMETERS HAS A BICYCLE?

As cyclists, we have an intuitive understanding of bicycle dynamics and their geometry. Most cyclists earned that understanding through scraped knees and bruises in their childhood and might never have given the theory of bicycle dynamics a thought. Still, we know how to steer and balance the bike by the handlebars or even without using our hands. When looking at an unknown bike we have a notion of what riding it might feel like. We know how different handling a dutch bike feels compared to handling a mountain bike and adjust our riding intuitively when we mount an unknown bike for the first time.

These points hint at important parameters and factors of bicycles and their balancing. The dutch bike feels different from a mountain bike primarily because they differ in their head angle, respective their front fork angle λ , as can be seen in Figure 2.1.

Even small changes to the front fork angle make a difference as “the results of the fine tuning of conventional bicycles to obtain the greatest suitability and handling for their intended purpose” [WSP20, pp. 365–366] show a difference of 2-3 ° between a touring and a racing bike.

The possibility of cycling hands-free and even driving through curves hands-free shows the importance of mass distribution and of the position of the center of gravity. While we cycle we constantly adjust our body’s center of gravity which in turn heavily influences the center of gravity C_g of the bike to keep the balance and keep the bicycle stable.

BACKGROUND



Figure 2.1: A urban racer versus a dutch bike with emphasis on their head angles

Additionally, when dealing with tilt we tend to employ another technique using the handlebars - mostly intuitively. We tend to steer in the opposite direction of the tilt and right the frame through the handlebar steering angle δ_f . This balancing by steering-control builds the basis of the modeling in this work, even if it is going to be implemented using motors instead of human hands.

Besides head angle, the center of gravity and the steering angle there are many more parameters that define a bicycle. We will focus on these which will later be used in our modeling and introduce them with their corresponding symbols.

To describe a bike and its characteristics we first need to establish a frame of reference to locate and orient its body in 3-dimensional space. The bike itself carries its own frame of reference, called the *body frame*, which moves with the bicycle in the larger *world frame*. Both are characterized by their three axes.

The world frame which will be used consists of the axes Z_{up} , Y_{north} , X_{east} , as can be seen in figure 2.2, and is oriented according to DIN ISO 8855.

2.1 WHAT PARAMETERS HAS A BICYCLE?

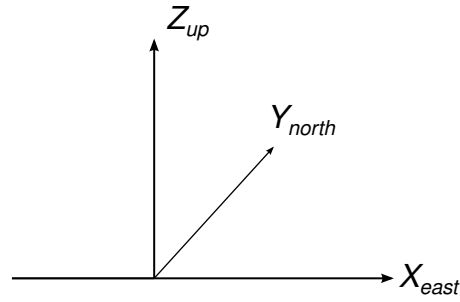


Figure 2.2: The world frame

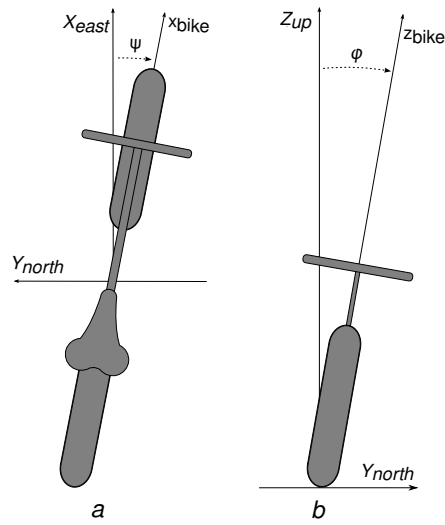


Figure 2.3: a) yaw angle ψ seen from above b) roll angle ϕ seen from the front

The bicycle frame itself carries the three axes of its body frame. The origin lies directly under the center of gravity on the horizontal plane. The z -axis goes from there straight up through the center of gravity C_g . Perpendicular lies the x -axis which dissects the bicycle in two. It lies on the contact points of the wheels. The y -axis lies perpendicular to both. The whole system and its reference frames are shown in figure 2.4.

The position of a bike can be represented as a 3-dimensional rotation between the two frames and is also called *attitude*. In reference to the bikes body frame movement around the z -axis is called **yaw**, around the y -axis **pitch**, and **roll** around the x -axis. Since for the constraints of this work there is no incline or decline of the xy -plane **pitch** is neglected and we will only work with **yaw** and **roll**. Their respective angles ψ and ϕ are illustrated in Figure 2.3.

To summarize, this work will make use of the following parameters:

BACKGROUND

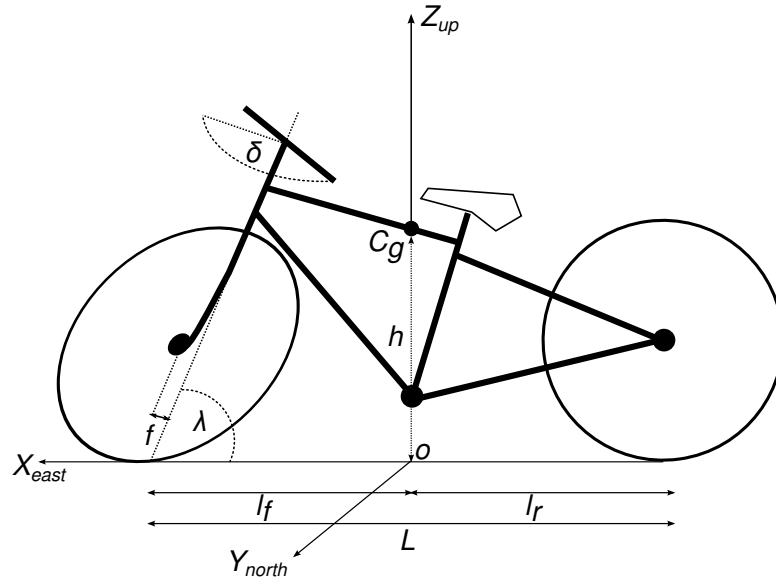


Figure 2.4: Bicycle Parameters

- The front fork angle λ , also called caster angle when measured from the vertical axis or head angle when measured from the horizontal axis, denotes the angle between the steering axis and the plane. The steering axis is defined by the head tube since this is where the first transmission of handlebar movement happens. In this work, only the head angle will be used so that for our purpose the fork angle and the head angle are one and the same.
- The roll angle ϕ which denotes the direction of movement around the x -axis
- The yaw angle ψ which denotes the direction of movement around the z -axis
- The handlebar steering angle δ
- The effective steering angle δ_f , which is the translated steering angle on the ground plane
- The center of gravity C_g , which for the sake of ease will carry the whole mass of the bicycle.
- The distance of the center of gravity to ground h

2.2 A VERY SHORT HISTORY OF BICYCLE MODELING AND SELF-BALANCING BICYCLES

- The mass m
- The fork offset f , which denotes the distance between the steering axis and the contact point of fork and wheel
- The wheelbase L , which denotes the distance between the contact points of the wheels. It is the sum of the distances l_r, l_f , which stand for the corresponding distance of the rear and front wheel contact points to C_g respectively

Now that the parameters are defined, let us take a look at how to use them in a model.

2.2 A VERY SHORT HISTORY OF BICYCLE MODELING AND SELF-BALANCING BICYCLES

As mentioned in the introduction, first attempts to model bicycles can be found in Whipple's [Whi19] and Carvallo's work [Car99] both in 1899. Whipple's bicycle model consists of four rigid bodies. The rear frame with or without a rider, the front frame which includes fork and handlebar and two wheels.

Its equations are linearized around the vertical equilibrium configuration, based on the Lagrangian dynamics. The model is still in use - even if often in modified form - today [Moo12].

Klein and Sommerfeld used the analysis of bicycles in their classic text about the physics of tops *Über die Theorie des Kreisels* [KS10].

The models of bicycles lend themselves to be used as a base for motorcycle modeling for their many similarities and are bundled into the category of *single track vehicles*. A lot of German research in the 20th century concentrated on increasing the stability of motorized single-track vehicles, as for example in Döhring's work *Die Stabilität von Einspurfahrzeugen* from 1955 [Döh55].

From then to now there have been numerous works about modeling bicycles and other two-wheelers. The number of works peaks about 1970, "perhaps because digital computers eased the integration of the governing equations[...]and because of an ecology-related bicycle boom" [Mei+07, p. 1957]. The same circumstances might be responsible for the contemporary peak of interest in bicycle research.

A good general introduction to the whole field is provided by the appropriately called book *Bicycling Science* [WSP20]. The article *Bicycles, Motorcycles, and Models* [Lim06] gives a good overview with a focus on control theory.

A more specific perspective is provided by *Linearized dynamics equations for the balance and steer of a bicycle: a benchmark and review* [Mei+07] in which the authors focus on modeling details, compare the different approaches and provide a benchmark.

Before continuing we need to define the difference between *self-stability* and *self-balancing*. In the context of this work *self-stability* may be defined as the phenomena that “a riderless bicycle can automatically steer itself so as to recover from falls” [Koo+11, p. 339], while *self-balancing* may be defined as a method of balance control of bicycles with the long-term goal of achieving autonomous driving of unmanned bicycles.

The study of self-stability of bicycles is still active and the models are far from complete. The continued effort shows that it is still not clear how self-stability works and what it depends on. This is very effectively shown by the experimental builds in *Bicycle Dynamics and Control: Adapted bicycles for education and research*[ÅKL05] and *A bicycle can be self-stable without gyroscopic or caster effects*[Koo+11].

The employed models range from non-linear models of varying order to even more complex multi-body systems.

An often employed model is that of the bike as an inverted pendulum, as for example in *Control for an Autonomous Bicycle*[GM95]. The authors use the underlying model of an inverted pendulum to design a controller which uses steering angle and rear wheel torque as input.

The inverted pendulum is a widely known model. As can be seen in Figure 2.12, the center of gravity is imagined atop a lever with the length h and therefore above its pivot point. Since the model stays in a 2D-plane and the movement of the lever is limited to $\pm 180^\circ$, simple trigonometry can be used to approximate the movement.

Besides the inverted pendulum, the popular *single-track model* can be used, which stems from the modeling of cars. A drawing can be seen in Figure Figure 2.13. The classic linear single track model allows the approximate, yet physically plausible, description of the lateral dynamics of a vehicle[SHB18]. The central idea for cars is to unite the front and rear wheels, per axle respectively, so that the vehicle has now only one track. This makes the model perfect for bicycles which are single track vehicles.

Both these approaches will later be used in the model of the prototype, as is described in section 2.7

The popularity of bicycle modelling and its use in teaching has led to it being the focus of several working groups at universities. Two examples are the bicycle dynamics group at TU Delft[Del21] and the bike research at Cornell’s Biorobotics and Locomotion Lab[Uni21].

Besides the vexing question of self-stability and new modelling approaches the designing and implementing of self-balance gains momentum as autopilots and autonomous driving are not exotic topics any longer.

Methods of keeping balance for self-balancing bicycles¹ classified according to [HZS15] and [Den+18] are the following:

¹ A certain subsection of self-balancing bicycles that is. Let us work by the definition that ordinary bike frame was used - ordinary in the sense of not custom built, but one similar available to these in the neighborhood bicycle shop. Purely experimental bicycles as these described by Åström[ÅKL05] or built by Kooijman[Koo+11] were not included.

2.2 A VERY SHORT HISTORY OF BICYCLE MODELING AND SELF-BALANCING BICYCLES

	CMG	Steering Control	Reaction Wheel	Mass Balancing
Advantages	Large Torque	Low energy consumption Real Time Response	Short Response Time	Simple Mechanical Structure
Disadvantages	High Energy Consumption	No Balance at low velocities	Limited output torque	Small Torque

Figure 2.5: Summary of advantages and disadvantages according to [Den+18]

- **Control Moment Gyroscope(CMG):**
A CMG possesses a spinning rotor in combination with one or more motorized gimbals which tilt the rotor's angular momentum. The changing angular momentum causes a gyroscopic torque which can counteract disturbances. Balance can even be achieved while the bicycle is stationary.
An example is presented by Yetkin et al.[Yet+14] in which a classic non-linear model with Lagrangian dynamics close to Whipple's model is employed. For controller design the theory of variable structure systems and sliding mode control theory is used.
- **Mass Balancing:**
A balancer can be imagined as a lever with an attached mass that can be shifted along the y_{bike} axes.
A theoretical elaboration can be found in Utano et al.[Uta+05].
- **Reaction Wheel:**
A reaction wheel is a type of flywheel. When changing its rotation speed disturbances can be countered by the conservation of angular momentum. Balance can even be achieved while the bicycle is stationary.
An example is Murata Boy by Murata Manufacturing[Co21]. Murata Boy is a robot on a bicycle. It can balance while driving forwards or backwards and even while standing. It uses a flywheel in its chest to stay balanced.
- **Steering Control:**
Steering Control makes use of the relationship between the steering angle and frame tilt. Its mechanical implementation ranges from transmission systems over steer-by-wire to linear actuators. Steering Control can only achieve balance if the bike is moving. Since this method will be used in the prototype, its mechanical implementation and examples thereof are discussed in more detail below.

Their advantages and disadvantages of the above methods are summarized in Figure 2.5

As the method of steering-control was chosen for this work its implementation needs to be determined. There are several mechanical implementations of steering-control in the literature. A selection is presented as follows:

- **Steer-by-wire:**
An example is presented by Tanaka and Murakami[TM04]. The

BACKGROUND

bicycle is modeled based on a dynamic model considering equilibrium of gravity and centrifugal force. The controller consists of a proportional and a differential controller. The results are simulated and measured in an experiment.

Unfortunately, the steer-by-wire approach by the authors seems only appropriate for simulations in a laboratory setting and not suitable for use in an autonomous bicycle that operates outside.

- **Linear Actuator:**

In his Master Thesis[Faw19] from 2019, Fawaz describes in detail the design and control of a self-balancing bicycle using an electric linear actuator. The bicycle is modeled and simulated using a second-order linear system. The control uses an proportional–integral–derivative(PID) controller.

While the linear actuator allows for exact control of the steering angle, it is very big and quite heavy.

- **Belt Transmission:**

An example is presented by He et al.[HZS15]. Using a classic non-linear model with Lagrangian dynamics close to Whipple’s model a constant-velocity steering controller is designed. The steering angle is controlled by a motor that steers via a toothed belt and gears. The resulting prototype can not only balance but easily turn.

- **Gear Transmission:**

Building on the above mentioned work by He et al.[HZS15] Deng et al. [Den+18] implement a steering mechanism consisting of gears that directly mesh with gears attached to the head tube and therefore do not need a belt. They further expand the prototype by adding a CMG.

As is later described in section 3.1, at first the steering-control for the prototype was implemented using direct gears before settling on a belt transmission.

The potential of the steering-control method is shown by an already quite polished prototype which was presented by TU Delft and Gazelle in 2019[Del19]. The bike can stabilize the rider by using steering control during velocities above $4 \frac{\text{km}}{\text{h}}$. The system allows normal steering by the rider, but corrects the steering angle slightly to achieve further balance.

All mentioned models and methods rely on capturing the bicycle’s position. How it can be measured will be discussed in the next section.

2.3 MEASURING POSITION

As we have seen, measuring position and state of the bike is central to controlling it. In section 2.1 two reference frames, the body frame and the world frame were established to orient the bike.

To capture its position the prototype makes use of an inertial measurement unit(IMU) which measures the bike’s specific force, its

2.4 WHAT IS A DC MOTOR AND ITS MOTOR CONSTANTS?

angular velocities and its orientation. The unit combines an accelerometer, a gyroscope and a magnetometer. Contemporary IMUs are often micro-electro-mechanical systems (MEMS) that employ tiny mechanical mechanisms inside an electronics package.

An accelerometer measures the sensor's acceleration in up to three axes. To illustrate how it measures we imagine the sensor sitting motionless on the ground. Let straight upwards towards the sky be the z -axis, to the right the x -axis and straight ahead the y -axis. The sensor will measure zero acceleration on the x - and y -axes, but earth's gravity $g = 9.81 \text{ m/s}^2$ on the z -axis. If the sensor falls straight down along the z -axis, it will measure zero acceleration along that axis since the acceleration of the fall is the exact opposite direction. Therefore raw measurements must always be adjusted to account for g .

A gyroscope measures the angular velocity along up to three axes. It can be imagined as a bicycle wheel. The rotational axis runs through the axle and the measured rotation is the wheel rotation. The sensor in the MEMS typically measures the deflection of a small mass by the Coriolis force.

A magnetometer measures the strength of the magnetic field along up to three axes. The classic compass is an example for a magnetometer that only measures direction. A MEMS magnetic field sensor typically makes use of the Lorentz force.

All of these sensors have their strengths and weaknesses. For instance, an accelerometer provides absolute measurements but also measures the additional acceleration if the sensor itself is moving without a way to differentiate the two. The magnetometer also gives absolute measurements but is easily disturbed by other magnetic influences and the Earth's magnetic field lines are not in a perpendicular position to the north pole. The gyroscope produces relative measurements and lacks initial orientation. Additionally, it suffers from drift over time due to the integration of inherent imperfections and internal noise.

To combat these weaknesses the IMU employs *sensor fusion*. The measurements of the three different sensors are combined to gain reliability and accuracy. While there are several sensor fusion algorithms the basics mostly stay the same. First, the outputs of accelerometer and magnetometer are combined to calculate the initial attitude of the device. Then the magnetic field readings and the gravity measurements are used to correct the gyro drift.

The IMU used in the prototype is described in detail in section 3.3.2.

2.4 WHAT IS A DC MOTOR AND ITS MOTOR CONSTANTS?

The prototype needs two motors, one for propelling the bike forwards, one for steering.

The steering motor is a *brushed* Direct Current (DC) Motor. This type of motor primarily consists of a rotor, the element which rotates, and the stator, which does not move [BKB19, p. 269 ff.]. The rotor

BACKGROUND

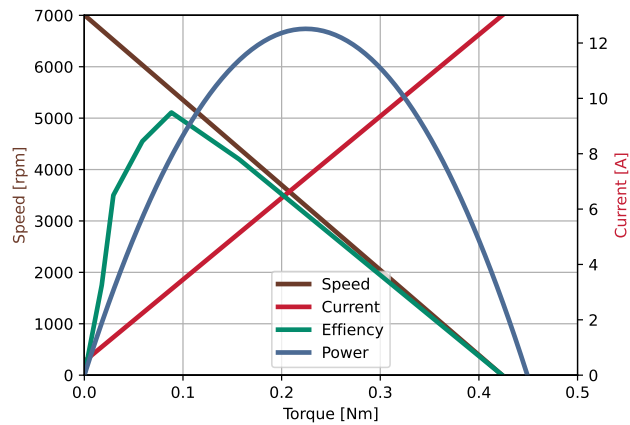


Figure 2.6: EMG49 Motor Characteristic[[Lim21a](#)]

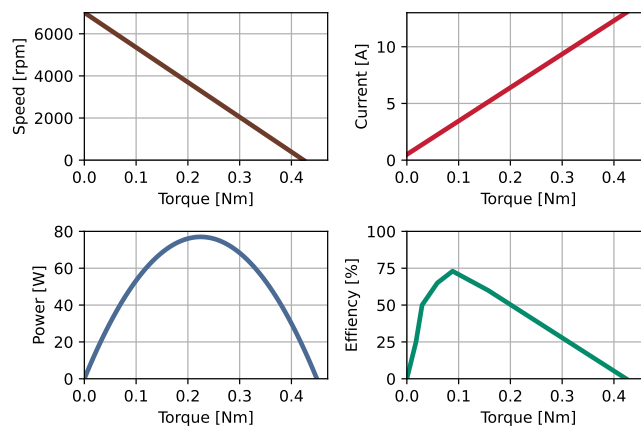


Figure 2.7: EMG49 Motor Characteristic[[Lim21a](#)]

consists of a shaft with attached brushes which are used to commutate the motor to cause it to spin. These wound coils which are supported by an iron core - or not if the motor is *coreless* - create a magnetic field.

The stator is fixed and wraps around the rotor. It usually consists of permanent magnets which provide another magnetic field. The motor is driven by building a magnet field through current supplied to the coils. This inner magnetic field has opposite polarity to the outer magnetic field. In order to create torque, which makes the rotor spin, the magnetic field of the rotor needs to continuously rotate. That means that the polarity of the inner field needs to change by switching the rotor current.

DC-motors are often compared by plotting their characteristics and calculating their motor constants. To illustrate we will look at the steering motors characteristic, to which we will come back to later in section 3.3.3.

The plotted characteristics can be seen in Figure 2.6 and separated into single graphs in Figure 2.7. The first of the single graphs, located in the upper-left, describes the relationship between speed and torque

and is intuitively comprehensible. Since the motor can only provide a limited amount of power P and $P = \tau \cdot \omega$, where τ is the torque and ω is the angular velocity², the two are inversely proportional to each other. The graph's zeros are the maximum velocity on the y - and the maximum torque on the x -axis.

The second graph in the upper-right of Figure 2.7 demonstrates that the relationship between torque and current draw is also linear, since $\tau = I \cdot K_t$, where τ is torque, I denotes the current and K_t is the torque constant to which we will come back in a bit.

The graph's zeros are the no-load current on the y - and the stall current on the x -axis. The term *stall current* is quite appropriate since it describes the point where the motor stops moving or at least can not accelerate any longer as it is under a heavy load.

In the lower-left of Figure 2.7 the power graph shows its typical parabola. Its vertex lies at exactly half the stall torque τ_{stall} where it reaches maximum mechanical power output. Next to it the efficiency curve shows how efficient the conversion from electrical power to mechanical power is, where $P_{electrical} = P_{mechanical} + P_{heat}$ and efficiency is defined as $100\% \frac{P_{electrical}}{P_{mechanical}}$.

Arguably the two most important constants for the prototype are the torque constant K_t , introduced above, and the velocity constant K_s . The velocity constant K_s describes the speed to voltage ratio and can be remembered as the inverse of K_t . Both constant's equation can be seen in equation 2.1.

$$\begin{aligned} K_t &= \frac{\tau}{I} \\ K_s &= \frac{1}{K_t} \end{aligned} \tag{2.1}$$

These constants express in one value what the designer of the motor put their focus on. The question if the motor is fast enough for the application given a set current and strong enough to turn the handlebar of the bike with only 24 V discussed with the help of the motor constants in section 3.3.3.

2.5 COMMUNICATION BETWEEN COMPONENTS

How do these different components communicate with each other? In the prototype, two protocols are used. The central unit communicates with the steering motor via a Universal Asynchronous Receiver-Transmitter(UART), while the IMU is controlled via the Inter-Integrated Circuit(I²C) protocol.

2 The angular velocity is represented in $\frac{\text{rad}}{\text{s}}$, while speed is mostly denoted in min^{-1} .

$$1 \frac{\text{rad}}{\text{s}} = \frac{60}{2\pi} 1/\text{min}$$

BACKGROUND

I^2C is an example of *serial* communication. The data is sent bit by bit, opposed to *parallel* communication where several bits are sent at the same time via several links. This means that serial buses usually have fewer lines, usually never more than four. I^2C needs two lines, the Serial Data Line(SDA) and the Serial Clock Line(SCL). The clock lines hint at another characteristic of I^2C - it is a *synchronous* protocol. That means that its transmission is timed and therefore synchronized by a clock.

A UART device can be seen as a sort of intermediary between serial and parallel communication. On the outside, e.g. on the actual two wires between the device and the sensor, the communication is purely serial. On the other side, the device is further connected by a at least 8-bit parallel bus. This way it can take a whole byte and send it in a serial fashion. The ports for the two lines are called Transmit(TX) and Receive(RX) and the lines need to go from (TX) to (RX) and (RX) to (TX).

Without going further into detail about these technologies - the important question is if they are fast and reliable enough to be used in the prototype.

I^2C has several modes and even in its slowest mode, the so-called standard mode, the transmission speed lies at $100 \frac{\text{kBit}}{\text{s}}$. The UART device on the motor controller communicates with a speed of $38.4 \frac{\text{kBit}}{\text{s}}$.

To check if the transfer rates are fast enough we need to look at the signal size. To get the current steering angle through function `get_steering_angle()`, described in section 3.5, 2 bytes need to be sent and 4 bytes received. That makes this a transfer of 8 bytes which should take about 8 ms. At an estimated moving speed of the bicycle at $10 \frac{\text{km}}{\text{h}}$ the bike would travel about 2 cm while the request and its answer are transferring. Although far from ideal it should suffice.

Now for the faster I^2C , even at its slowest mode. The function `get_eul()`, described in section 3.5, which returns a full set of euler angles sends 1 byte and reads 6, for a total of 56 bit. Its transfer would only take up to 0.5 ms which is sufficient.

Of course, these two times are a rough estimate but nevertheless show that the UART might become a problem at slightly higher speeds but it is sufficient, for the current scope.

2.6 CONTROLLING THE SYSTEM

Since it is our goal to control the bike we need to think about control. One may think it may be enough to let the bicycle keep a constant steering angle and let the motor drive it forwards. As long as we have a good model and there are no disturbances, this might go well. Unfortunately, our model and predictions still being far from perfect, as described in section 2.2. Additionally, we would need to perfectly model any disturbance that may influence the bike. Since modeling all these is far too involved we tend to rely on *feedback*. That means, we can “define control to be the use of algorithms and feedback in engineered systems” [ÅM21, p. 5].

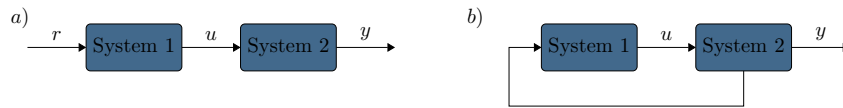


Figure 2.8: a) Open loop system b) Closed loop system

Let us come back to the first idea of a constant steering angle. It falls under the category of so-called *open-loop systems*³ whose structure can be seen in Figure 2.8.

An open-loop can easily be implemented as long as the involved system is not complex. A simple example is a flashlight with a switch - at least, if we ignore the most complex system, the human who operates the switch. Its block diagram can be found in Figure 2.9. Our *input* is the switch position that controls the *output* - the light. The output is not influenced by any other circumstances - neither if the surroundings are already bright enough because of daylight, nor if its light is too bright because it is pitch black - only the switch position counts. In control theory terms, the system is ignorant of any *disturbances* that are influencing its output.

This simplicity makes open loop systems unsuitable for even only slightly more complicated tasks. A typical example is temperature control. Imagine that we want a certain temperature in our room while using an electric heater. For the sake of brevity, let us assume that the heater produces more heat the more current is supplied. We all know, that the effective temperature heavily depends on the temperature of the surrounding environment. In winter the heater needs to generate a lot more heat to reach the desired 23 °C than in spring.

How can we make the system respond to its environment? With a sensor of course. Only if the system measures the actual temperature it can regulate its heat output to get closer to the desired temperature. The measurement is what introduces *feedback*. “The principle of feedback is simple: base correcting actions on the difference between desired and actual performance” [ÅM21, p. 13]. This kind of system falls under the category of *closed-loop systems*, whose structure can be seen in Figure 2.8. The heater system needs a new instance that is able to compare the measurement with the desired input and regulates the current to the heater which in turn determines the heat output. As can be seen in Figure 2.9, this instance is a *controller*.

Please note the little minus sign next to the big plus sign in Figure 2.10. This means that it is *negative feedback* and the *error* is calculated as

$$e = \Delta t = t_{input} - t_{measure}$$

As the difference between the desired temperature t_{input} and the measured temperature t_m .

³ A note to german readers: This work will stick to the vocabulary used in [ÅM21]. *Open-loop system* corresponds to the german *Steuerung*, while *closed-loop system* corresponds to the german *Regelung*

BACKGROUND

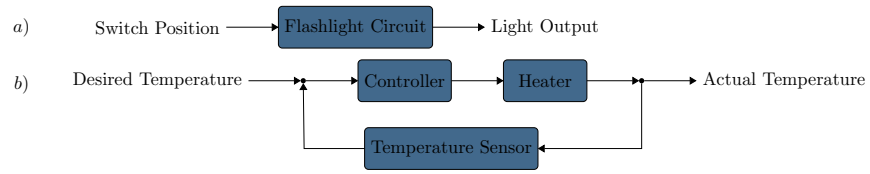


Figure 2.9: Block diagram of a) flashlight and b) heater

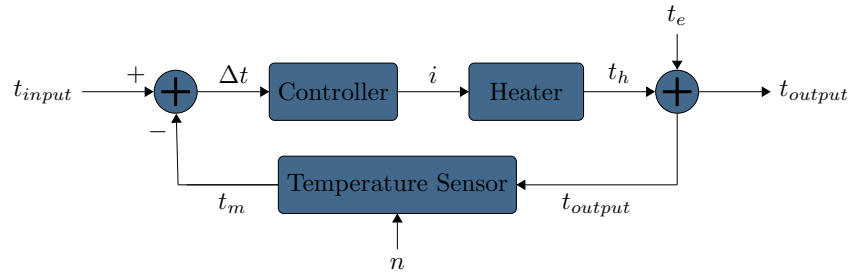


Figure 2.10: Heater System Detail

Would it be *positive feedback* the system would get *unstable*. *Stability* is an important characteristic of a controller. An unstable controller begins to self-oscillate and can not be saved. We are all familiar what that sounds like since that is the reason bad audio feedback sounds so terrible.

Another detail of Figure 2.10 is the influence of *noise* n . Since there is no perfect sensor, the measured value $t_m \neq t_{output}$, but $t_m = n(t_{output})$. How well the controller can deal with deviations from the used model, disturbances and noise is expressed in its *robustness*.

But what does the controller do with the error? The most basic of all controllers is the so-called *proportional controller*. In the above example the controller would apply the error and a coefficient k_p to the current output - as long as the error is inside an expected interval, the *linear range* (e_{min}, e_{max}) . Let $u(x)$ be the function that describes the controller

$$u(x) = \begin{cases} u_{max} & \text{if } e \geq e_{max} \\ k_p e & \text{if } e_{min} < e < e_{max} \\ u_{min} & \text{if } e \leq e_{min} \end{cases}$$

The coefficient k_p is typically a numerical value < 1 and is mostly defined by experimentation. It is often the first step in implementing a controller and is used for the prototype, too.

One step up from proportional controllers are the *integral* and the *derivation* controllers. As their names suggest there is integration over or derivation of the error.

We have seen, stability is a central characteristic of a system. Ideally, we want to be able to calculate if our designed system is stable. In

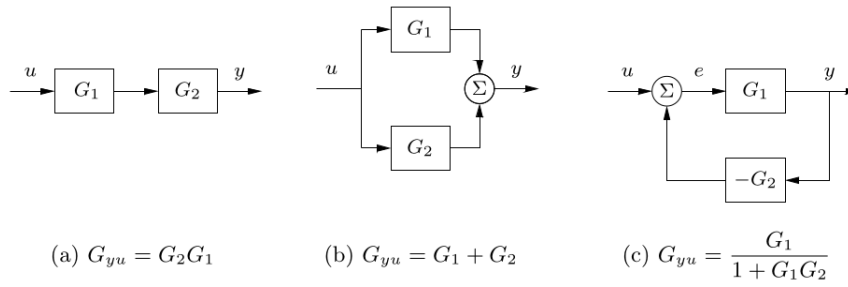


Figure 2.11: Interconnections of linear systems. Series (a), parallel (b), and feedback (c) connections are shown [ÅM21, p.9-17]

control theory, so-called *transfer functions* are used to model the system in the frequency domain.

Why the frequency domain? Many of the processes we implement depend on time. In the time domain, it can be difficult or even impossible to observe certain characteristics of a signal, e.g. cyclic behavior. The frequency domain representation allows for easier interpretation of the signal's behavior.

The transfer function $G(s)$ of a function $f(t)$ in the time domain can be obtained by using the Laplace Transform on $f(t)$, so that

$$G(s) = \mathcal{L}_t\{f(t)\}$$

Changing to the frequency domain has another advantage. A control loop often consists of one or more blocks. The open-loop system in Figure 2.8 shows two block in series. System 1 is $f_1(r) = u$ and System 2 $f_2(u) = y$. To model the output function $y(t)$ with any input $u(t)$ we would need to find the convolution

$$y(t) = u(t) * f_1(t) * f_2(t)$$

which would result in a quite unwieldy convolution integral. A convolution in the time domain is a multiplication in the frequency domain, which makes the simulation a lot more pleasant. Further rules of the combination of different blocks of transfer functions are shown in Figure 2.11

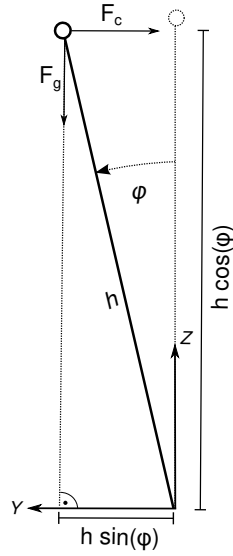


Figure 2.12: Tilt of the frame modeled as inverted pendulum

2.7 AND HOW CAN WE MODEL THE PROTOTYPE?

So how can we model our goal? The balance of a bicycle can be described as having a roll angle $\phi = 0$. As already mentioned in section 2.1 tilt can be described through the roll angle ϕ and - naively - a balanced system should therefore have it at 0.

We have seen in section 2.2 that there is more than one way to model a bicycle. Since the used models only need to be a proof of concept for the prototype, two basic models are used to model the relationship the steering angle δ and the roll angle ϕ .

The first is the model of an *inverted pendulum* and the second the *single-track model*, as already mentioned in section 2.2.

Both depend on the following base assumptions, as per [HZS15] and [SHB18]:

- The bicycle is regarded as a rigid body.
- The wheels of the bicycle do not side-slip.
- The velocity in the center of mass is considered constant along its trajectory.

The inverted pendulum is a widely known model. As can be seen in Figure 2.12, the center of gravity is imagined atop a lever with the length h and therefore above its pivot point. Two forces act on C_g

$$\begin{aligned} F_g &= mg \\ F_c &= ma_y \end{aligned} \tag{2.2}$$

2.7 AND HOW CAN WE MODEL THE PROTOTYPE?

where m is the mass, g is the gravity and a_y is the lateral acceleration. As per the single-track model, the lateral a_y acceleration during constant velocity v can be assumed to be $a_y = \frac{v^2}{\rho}$, where ρ is the radius [SHB18, p.107]. The radius ρ is assumed as

$$\rho = \frac{l_r}{\delta} \quad (2.3)$$

because of the so-called *Ackermann steer angle*, described in [SHB18, p.226 ff.]

These forces F_g, F_c - together with their respective lever arms - form two torques

$$\begin{aligned} M_g &= F_g h \sin(\phi) \approx mgh\phi \\ M_c &= F_c h \cos(\phi) \approx ma_y h = m \frac{v^2}{\rho} h \end{aligned} \quad (2.4)$$

which are approximated for expected small ϕ .

To now form a dynamic equation we need to describe the inertia J_1 of the bike, since it counteracts the torques we just described. We can express it as the mass moment of inertia

$$J_1 = mh^2\phi \quad (2.5)$$

There is still another movement we have not accounted for. Unfortunately, the bicycle not only tilts but can be assumed as moving along a curve at the same moment, which equals a rotation around the z_{bike} axes. This rotation leads to another inertia J_2 along the x -axes of the pendulum.

$$J_2 = ml_r h^2 \dot{\omega} \phi \quad (2.6)$$

where l_r is the distance between the rear wheel and C_g projected onto the x -axis and ω is the angular velocity.

$$\omega = \frac{v}{\rho} \delta \rightarrow \dot{\omega} = \frac{v}{\rho} \dot{\delta} \quad (2.7)$$

This lets us form the equation

$$\begin{aligned} J_1 + J_2 &= M_g - M_c \\ mh^2\phi + ml_r h^2 \dot{\omega} &= mgh\phi - m \frac{v^2}{\rho} h \end{aligned} \quad (2.8)$$

BACKGROUND

Using equations 2.3 and 2.7 we derive

$$mh^2\ddot{\phi} + ml_r h^2 \frac{v}{l_r} \dot{\delta} = mgh\phi - m \frac{v^2 \delta}{l_r} h \quad (2.9)$$

The above equation matches the equation provided by Åström et al. [ÅKL05, eqn.1, p.29]. That not only means that our calculation seems to be correct but that we can rely on its poles, zeroes and transfer function. Rewriting in our notation yields the following poles p_1, p_2 and zero z .

$$\begin{aligned} p_{1,2} &= \sqrt{\frac{mgh}{J_1}} \sqrt{\pm} \sqrt{\frac{g}{h}} \\ z &= -\frac{mvh}{-J_2} \approx \frac{V}{a} \end{aligned} \quad (2.10)$$

Since the function has more poles than zeroes, it can be considered proper. The transfer function from steering angle δ to roll ϕ according to Åström et al. [ÅKL05, eqn.4, p.29] is

$$\begin{aligned} G_{\phi,\delta}(s) &= \frac{v(-J_2 s + mVh)}{b(J_2 s^2 - mgh)} \\ &= \frac{-vJ_2}{bJ_1} \frac{s - \frac{mVh}{-J_2}}{s^2 - \frac{mgh}{J_1}} \approx \frac{l_r v}{Lh} \frac{s + \frac{v}{l_r}}{s^2} - \frac{g}{h} \end{aligned} \quad (2.11)$$

The authors further remark that is unstable and needs to be stabilized by

$$\delta = -k_\phi \phi \quad (2.12)$$

to yield the closed-loop system[ÅKL05, eqn.6, p.29]

$$J\ddot{\phi} - \frac{J_2 v k_\phi}{L} \dot{\delta} + \left(\frac{mv^2 h k_\phi}{L} - mgh \right) \phi = 0 \quad (2.13)$$

which is only asymptotically stable when

$$k_{phi} > \frac{Lg}{v^2} \quad (2.14)$$

Both the equations 2.12 and 2.14 provide valuable insight into modelling the feedback and its coefficient k_ϕ in the software implementation. Assuming velocities of $2 \frac{\text{m}}{\text{s}}$ and $3 \frac{\text{m}}{\text{s}}$ and $g = 9.81 \text{ m/s}^2$ with the

2.7 AND HOW CAN WE MODEL THE PROTOTYPE?

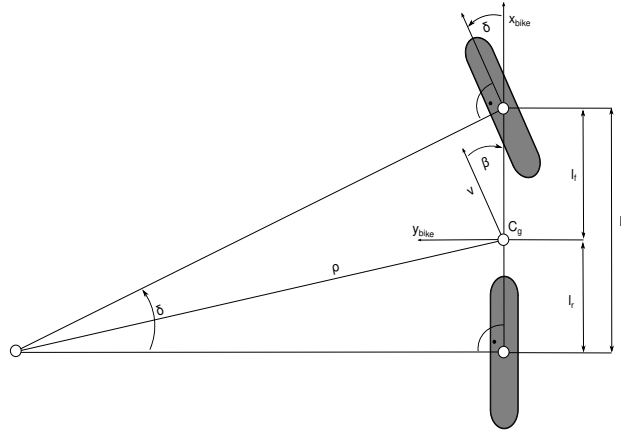


Figure 2.13: Single track model of the bicycle with the center of gravity as reference point

measured wheelbase $L = 1.05$ m, see Figure 3.3, yields approximate values of

$$\begin{aligned} k_{phi} &> 2,58 \quad \text{for } v = 1 \frac{m}{s} \\ k_{phi} &> 0,29 \quad \text{for } v = 3 \frac{m}{s} \end{aligned} \quad (2.15)$$

Since the bicycle prototype needs to be pushed in the beginning and walking velocities are in that range the above set the limits for the initial k_{phi} in section 3.5.

Additional models of the used toothed belt and the steering motor control are discussed in sections 3.2 and 3.3.3, respectively. The whole control loop is finally discussed in section 3.4.

Since this is the second model, the first is documented in appendix A and its pitfalls are discussed in section 4

SYSTEM DEVELOPMENT



Figure 3.1: The complete platform being tested

The working group wished to use bicycles to demonstrate the implementation of control-systems and to serve as an experimental platform. Students would be able to compare different control algorithms in practice and measure the corresponding models' accuracy. At the start of experimenting the goals for the first platform prototype were set as follows:

- It should be reproducible and use standard parts:
The bicycle should use an everyday frame to make the topic more accessible. Easily sourced parts guarantee reproducibility and keep the cost for replacements inexpensive.
- It should be easily expandable:

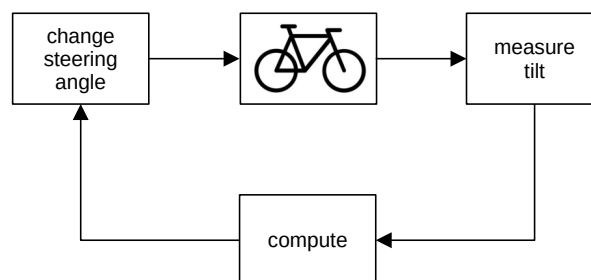


Figure 3.2: Steering Control System Diagram

Parameter	Symbol	Value
front fork angle	λ	67 °
fork offset	f	0.07 m
mass	m	27 kg
distance C_g	h	0.7 m
wheelbase	L	1.05 m
front wheel to C_g	l_f	0.60 m
rear wheel to C_g	l_r	0.45 m

Figure 3.3: Static parameters of the chosen bicycle

The system should be powerful enough to accommodate a future machine-vision project as well as provide plenty of connectivity for more sensors and/or actors.

- It should favor open-source hardware and software:
The use of open-source may prevent the future disappearance or missing adaptability of used software. Students will be able to fork used software if extensive adaption is desired. The available information on hardware that is open-source enables students to familiarize themselves in detail with the used architecture and promotes further exploration of the used technologies.
- It should use an electric drive:
To guarantee constant velocity for experimenting and measuring an electric drive is preferred. Besides the practical use the perspective of an autonomous bicycle may serve as a big motivator for prospective students.
- It should use steering-control to balance:
The initial literature review showed the inexpensiveness and intuitiveness of balance by steering-control compared to other avenues, as described in section 2.2.

As with every bike, it all started with a frame.

3.1 THE BIKE

Since the project's goals call for the use of standard parts an everyday mountain bike was chosen as the base. It has 26" wheels which facilitate the handling and its storage. The static parameters of a bike frame are not that easily measured and the table in Figure 3.3 can be seen as an approximation. The provided mass includes all added parts and the batteries. Since the position of C_g could not be measured precisely, its position was set at the tilt sensor's position which benefits the modeling.

As electric drive, a motorized rear wheel was chosen. It features an 48 V, 1500 W brushless non-gear hub motor [Ltd21b] and has its own controller which has not been altered in this first prototype. Since the motor is quite powerful the bike quickly accelerates to about 30 $\frac{\text{km}}{\text{h}}$.

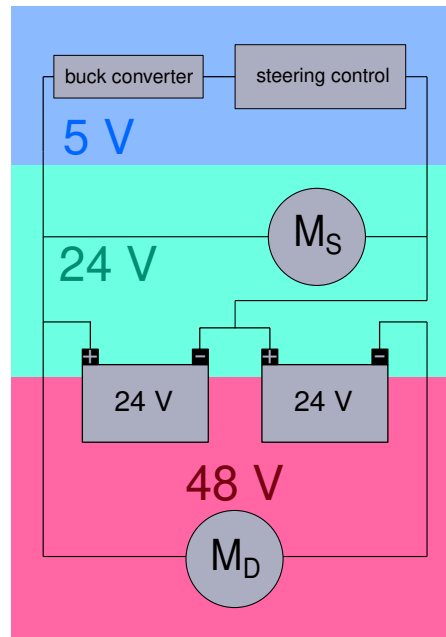


Figure 3.4: The supply voltages

The manufacturer quotes the full no-load speed at about $50\text{--}60 \frac{\text{km}}{\text{h}}$. Therefore higher velocities seem possible but remain to be tested.

Motor power is controlled via a thruster at the handlebar which consists of a simple hall sensor. The motor control is connected via a simple switch to the brake levers and cuts the motor current if one of the brake levers is pulled.

The driving motor is supplied by two batteries with 24 V each. The batteries are connected in series and supply the motor with the required 48 V. Between the batteries and the rest of the system an emergency stop and a pull switch were installed to address safety concerns. These parts, as all parts installed, are listed in Appendix D

The same batteries supply the power for the whole system. An overview of all power stages can be found in Figure 3.4. The 24 V power stage is supplied by a single battery and a DC-to-DC converter provides the 5 V power stage.

3.2 THE MECHANICS OF THE STEERING-CONTROL

After acquiring the frame, power supply and driving motor the steering-control had to be implemented. The first approach used 3D-printed gears whose model can be seen in Figure 3.5.

The steering-motor, described in section 3.3.3, sat attached to the front fork and drove the input gear with 32 teeth which in turn drove the output gear with 72 gears. The output gear consisted of two halves that wrapped around the head tube.

The gears were printed in polylactic acid(PLA) and later in polyethylene terephthalate glycol(PETG) using Fused Deposition Modeling(FDM) 3D-printing.

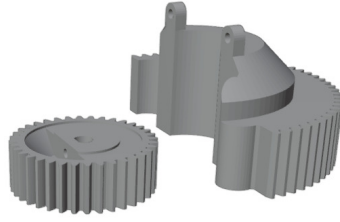


Figure 3.5: The gears of the first approach



Figure 3.6: The final version uses a toothed belt and two gears to steer

The correct positioning of the motor proved quite difficult. The gears tendency to jam was aggravated by the printed surface. After several prototypes jammed up and broke a new approach was needed.

To relax the exact positioning a belt drive was proposed and a timing belt with a T5 profile chosen. The input gear G_A with $N_A = 30$ teeth sits on the steering motor and drives the output gear G_B which has $N_B = 44$ teeth, as shown in Figure 3.6.

The bigger gear is fastened to the handlebar stem via a taper bush while the input gear is fastened on the drive shaft with a socket screw. To bring the belt to tension a custom vise with a 3D-printed steering motor holder was built and attached to the side of the bike as can be seen in Figure 3.1.

To prepare for controlling the system, documented in section 3.4, we need the following definition of the gear transmission ratio ι

$$\iota = \frac{n_A}{n_B} = \frac{\omega_A}{\omega_B} = \frac{N_B}{N_A} \quad (3.1)$$

where n_A, n_B are the rotation speeds, ω_A, ω_B the angular velocities and N_A, N_B the number of teeth of the respective gears G_A, G_B . The whole transmission is illustrated in Figure 3.7.

Especially important is the steering motor angle γ_A and its relation to the handlebar steering angle δ which equates to γ_B .

$$\gamma_A = \iota \gamma_B = \frac{N_B}{N_A} \delta \quad (3.2)$$

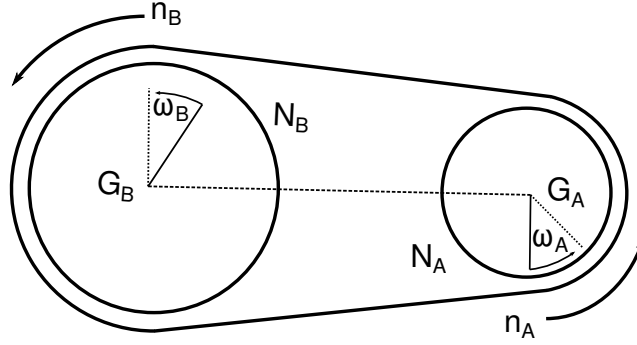


Figure 3.7: Gear transmission of steering motor to handlebar steering angle

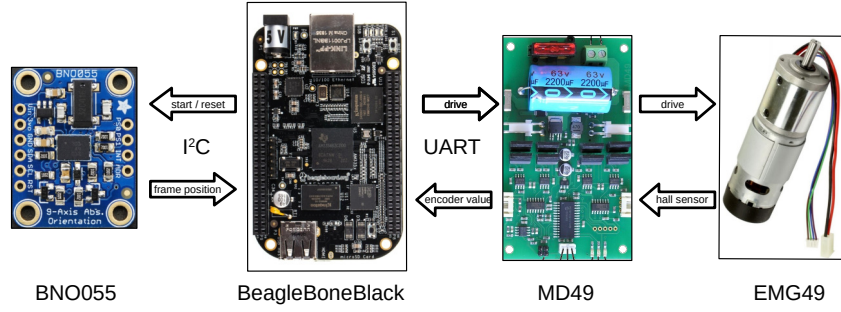


Figure 3.8: Steering Control Hardware

The rotational velocities ω_A, ω_B are related similarly

$$\omega_A = \omega_B = \frac{N_B}{N_A} \omega_B \quad (3.3)$$

For the sake of model simplicity disturbances such as belt slack and inertia will be ignored. One thing to keep in mind is that the rotation speed n of the steering motor is given as 1/min and angular velocity ω in $\frac{\text{rad}}{\text{s}}$ which can be converted as follows

$$\omega = n \frac{2\pi}{60} \quad (3.4)$$

3.3 THE ELECTRONICS OF THE STEERING-CONTROL

As can be seen in Figure 3.8 the central unit consists of a BeagleBone Black which drives a EMG49 motor through a MD49 motor controller board and polls position data from a BNO055 IMU.

3.3.1 *BeagleBone Black*

The BeagleBone Black[Fou20] is an open-source singleboard-computer. It features an AM3358¹ System-on-Chip(SoC) by Texas Instruments[Ins21] which includes an 32-bit 1 GHz Arm Cortex-A8 central processing unit(CPU).

With 69 general-purpose input/output(GPIO) pins it provides ample connectivity and surpasses its most famous competitor, the Raspberry Pi 4, which only provides 40 GPIO pins[Fou21b].

The full schematics, bill of materials(BOM) and even Gerber files are available in the Foundation's repository[Fou21a]. Several operating systems ports are available. Most are Linux distributions but Android and BSD ports are also available.

Attractive for the use in robotics is the availability of a fitted Robot Operating System and Real-Time-Operating-Systems like FreeRTOS and TI-RTOS for real-time(RT) applications.

This brings us to a particularity of the AM3358 which was central for choosing the BeagleBone Black as the central part. The SoC features two Programmable Real-Time Unit Subsystem and Industrial Communication Subsystems(PRU-ICSS) co-processors. These make it possible to measure in real-time. The future analysis of models, as described in the project goals, can only profit from this possibility. Its future use is discussed in section 4.3.

Besides using the PRU-ICSS, Texas Instruments provides a RT-Kernel and complete images with this kernel are available from the BeagleBone Foundation. While not as exact as using the PRU-ICSS, making use of the PREEMPT_RT kernel patch might be perceived easier.

Further details are documented in the context of the software implementation and can be found in section 3.5.

3.3.2 *BNO055*

As discussed in section 2.3 the system must be able to sense its position and movements. Chosen as the sensor was the Adafruit 9-DOF Absolute Orientation IMU Fusion Breakout[Ind21], as can be seen in Figure 3.9 which features the Bosch BNO055[Sen20]. It includes an accelerometer, gyroscope and magnetometer and brings its own sensor fusion on-chip.

It communicates with the BeagleBone Black via I², which is discussed in section 2.5.

That the sensor brings its own sensor fusion has its advantages and drawbacks. One clear advantage is that a complete set of Euler angles is provided as output. How they are formed remains Bosch's secret since their sensor fusion algorithm is not public.

The calibration of the IMU is rather involved. Even though a step-by-step description[Sen20, p. 51 ff.] of the calibration for accelerometer and gyroscope is provided and the process' result can be saved in a

¹ to be more specific the used Revision C features a Sitara AM3358BZCZ[Fou20]

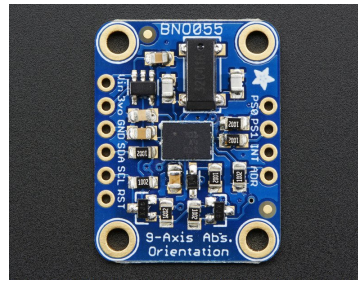


Figure 3.9: Adafruit 9-DOF Absolute Orientation IMU Fusion Breakout[[Ind21](#)]

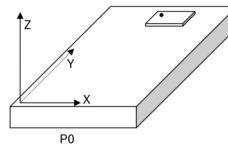


Figure 3.10: Remapping scheme $P0$ for the BNO055 chip [[Sen20](#), p. 27]

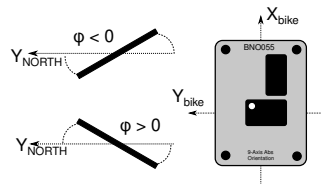


Figure 3.11: Orientation of roll angle after axes remapping

calibration profile the process is far from smooth. It can take a very long time for the `CALIB_STAT` registers to signal significant calibration without any apparent reasons. One possible cause could be the automatic calibration is always running in the background. It can not be disabled and its inner workings stay a mystery.

A nice feature of the BNO055 is the possibility of axes remapping which enables us to fit the sensor axes to the bike. The most fitting scheme seems to be $P0$, which is shown in Figure 3.10. The corresponding roll angle and its orientation after the remap are shown in Figure 3.11

Its direct use is further documented in the context of the software implementation and can be found in section 3.5. As the calibration process proved so erratic that the calibration check was removed from the software implementation a possible alternative approach is discussed in section 4.3.

3.3.3 MD49 and EMG49

The drive system for the steering control uses the MD49 motor controller board[[Lim21b](#)] and an EMG49 DC-motor[[Lim21a](#)] from Devan-

3.3 THE ELECTRONICS OF THE STEERING-CONTROL

Parameter	Units	Value
Nominal voltage	V	24
No-load speed internal	rpm	7007
No-load speed external	rpm	143
No load current	A	0.5
Rated speed	rpm	122
Rated torque	N m	0.1569
Rated current	A	2.1
Stall torque	N m	0.431491
Starting/Stall current	A	13
Max Power	W	77
Max Efficiency	W	73
K_s	$\frac{\text{rpm}}{\text{V}}$	5,96
$K_t = K_m$	$\frac{\text{Nm}}{\text{A}}$	73
K_e	$\frac{\text{V}}{\text{rpm}}$	73

Figure 3.12: EMG49 Properties[[Lim21a](#)], entries in red were approximated from the vendor-supplied information

tech. The system uses a 24 V supply and a logic level of 5 V. The controller board communicates with the BeagleBone Black via UART, which is explained in section 2.5, using a logic level shifter.

As already mentioned in section 2.4, a motor can be described by its characteristics. Not all were provided by the supplier. In Table 3.12 the provided and approximated characteristics of the motor are listed.

The approximated characteristics were read from the provided curves and calculated according to the following equations. First the speed constant K_s was calculated by using the no-load speed ω_{noload} and the nominal voltage V .

$$K_s = \frac{\omega_{noload}}{V} = \frac{143 \text{ rpm}}{24 \text{ V}} \approx 5.96 \frac{\text{rpm}}{\text{V}} \quad (3.5)$$

To determine the stall constant K_t the stall torque $\tau_{stall} = 0.1569 \text{ N m}$ is read from the provided curves in Figure 2.7, since it is the maximum intercept point of the x -axis of all graphs except the Current/Torque-Graph where the graph lies on the point $(\tau_{stall}, i_{stall})$.

Then the stall constant K_t can be determined by

$$K_t = \frac{\tau_{stall}}{i_{stall}} = \frac{0.1569 \text{ N m}}{13 \text{ A}} \approx 0.012 \frac{\text{N m}}{\text{A}} \quad (3.6)$$

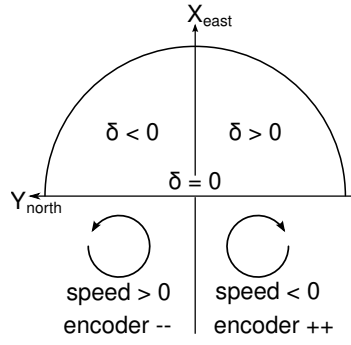


Figure 3.13: Variables of motor control

The EMG49 uses a hall sensor to determine the shaft position and an encoder counts 980 encoder turns per output shaft turn. This leads to an encoder count E_{motor} to angle γ_E constant $\Gamma_{Encoder}$, which will be used in the control loop in section 3.4.

$$\Gamma_{Encoder} = \frac{2\pi\text{rad}}{980} \quad (3.7)$$

The direction of the values is illustrated in Figure 3.13.

3.4 THE CONTROL LOOP

With the background of control loops in section 2.6 and the model of the prototype in section 2.7 we can complete the model of our control loop. The whole loop can be seen in Figure 3.14.

The goal to balance the bicycle can be modeled as $\phi_{input} = 0$ which describes the desired value of the roll angle to be zero. This is fed into addition with the feedback, which provides the measured roll angle $\phi_{measured}$. The error $e_\phi = \Delta\phi = \phi_{input} - \phi_{measured} = \phi_{measured}$ goes to the first block, the proportional controller.

$$f_1(e_\phi) = k_\phi e_\phi = k_\phi - \phi_{measured} = \delta_0 \quad (3.8)$$

The feedback therefore satisfies the constraint $\delta = k_{phi}\phi$ from equation 2.12. We need to keep the calculated range of the proportional coefficient k_ϕ from equation 2.15 in mind for the later implementation in section 3.5.

The angle δ_0 needs to be transformed to the drive gear angle γ_A , as establish in equation 3.2. This is a linear operation with that depends on the transmission ratio i , defined in equation 3.1

$$f_2(\delta_0) = \frac{N_B}{N_A} \delta_0 = \gamma_0 \quad (3.9)$$

The angle needs to be expressed in encoder turns, so the motor loop can be entered. This is a linear operation with the constant $\Gamma_{Encoder}$, defined in equation 3.7.

$$f_3(\gamma_0) = \Gamma_{Encoder} \gamma_0 = E_\gamma \quad (3.10)$$

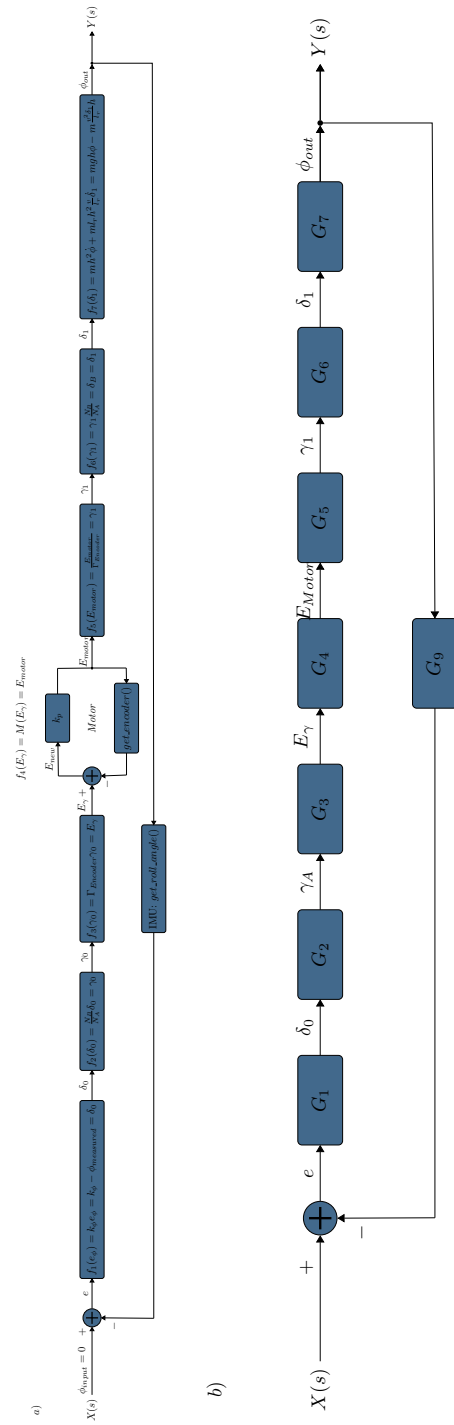


Figure 3.14: The Control System Block Diagrams *a*) in the time domain *b*) in the frequency domain

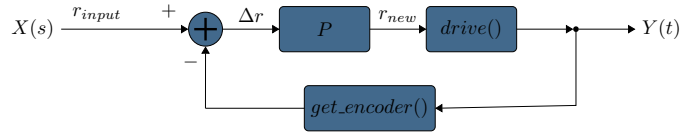


Figure 3.15: The Motor Control Block Diagram

The desired encoder count is the input for the following motor loop. The motor loop possesses its own feedback loop with its proportional controller $P_m = k_{pm}e_m$. The motor loop can be seen isolated in Figure 3.15. Since it carries its own feedback it can later be included by using its transfer function. For now, its function is just a placeholder.

$$f_4(E_\gamma) = M(E_\gamma) = E_{motor} \quad (3.11)$$

After the motor loop we transform the encoder turns output E_{motor} by the motor loop into the angle of the steering motor shaft and therefore the driving gear G_A by employing the constant $\Gamma_{Encoder}$ again.

$$f_5(E_{motor}) = \frac{E_{motor}}{\Gamma_{Encoder}} = \gamma_1 \quad (3.12)$$

The relation between the angles γ_A, δ_B is defined by the gear transmission ratio and was established in equation 3.1

$$f_6(\gamma_1 make) = \gamma_1 \frac{N_B}{N_A} = \delta_B = \delta_1 \quad (3.13)$$

The difference between the prior δ_0 and δ_1 is that the second carries eventual errors on the motor loop. With δ_0 we can use the dynamic equation 2.9, which was derived by combining the model of an inverted pendulum and the single-track-model

$$f_7(\delta_0) = mh^2\ddot{\phi} + ml_r h^2 \frac{v}{l_r} \dot{\delta} = mgh\phi - m \frac{v^2 \delta}{l_r} h \quad (3.14)$$

its output is the projected roll angle ϕ_{out} .

After formulating the single functions in the time domain and orienting them in the block diagram, Figure 3.14, we can continue working towards the frequency domain model.

To model the system in the frequency domain above functions need now be transformed via Laplace transform, as described in section 2.6.

The input as a function of time and later frequency is modeled as

$$\begin{aligned} \phi_{input}(t) &= 0 \\ \phi_{input}(s) &= \mathcal{L}_t\{f_1(t)\} = 0 \end{aligned} \quad (3.15)$$

The proportional gain then is modeled as

$$G_1(s) = \frac{\delta_0(s)}{e(s)} = k_\phi \quad (3.16)$$

The transfer from δ_0 to the drive gear angle γ_A

$$G_2(s) = \frac{\gamma_A(s)}{\delta_0(s)} = \frac{N_B}{N_A} \quad (3.17)$$

To enter the motor loop, the multiplication with the constant $\Gamma_{Encoder}$

$$G_3(s) = \frac{E_\gamma(s)}{\gamma_A(s)} = \Gamma_{Encoder} \quad (3.18)$$

The motor loop carries its own feedback and proportional controller, as can be seen in Figure 3.15. It is therefore modeled as shown in Figure 2.11.

$$G_4(s) = \frac{E_{Motor}(s)}{E_\gamma(s)} = \frac{k_{pm}}{1 + k_{pm}} \quad (3.19)$$

Exiting the motor loop requires the constant $\Gamma_{Encoder}$ again

$$G_5(s) = \frac{\gamma_A(s)}{E_{motor}(s)} = \frac{1}{\Gamma_{Encoder}} \quad (3.20)$$

Now for the transmission again

$$G_6(s) = \frac{\delta_1(s)}{\gamma_A(s)} = \frac{N_B}{N_A} \quad (3.21)$$

Finally the big transfer function from equation 2.11

$$\begin{aligned} G_7(s) &= \frac{v(-J_2s + mVh)}{b(J_2s^2 - mgh)} \\ &= \frac{-vJ_2}{bJ_1} \frac{s - \frac{mVh}{J_2}}{s^2 - \frac{mgh}{J_1}} \approx \frac{l_r v}{Lh} \frac{s + \frac{v}{l_r}}{s^2} - \frac{g}{h} \end{aligned} \quad (3.22)$$

$$\begin{aligned} G_{all} &= \frac{G_7 G_6 G_5 G_4 G_3 G_2 G_1}{1 + G_7 G_6 G_5 G_4 G_3 G_2 G_1} \\ &= \frac{\frac{l_r v}{Lh} \frac{s + \frac{v}{l_r}}{s^2} - \frac{g}{h} \frac{N_B}{N_A} \frac{1}{\Gamma_{Encoder}} \frac{k_{pm}}{1 + k_{pm}} \Gamma_{Encoder} \frac{N_B}{N_A} k_\phi}{1 + \frac{l_r v}{Lh} \frac{s + \frac{v}{l_r}}{s^2} - \frac{g}{h} \frac{N_B}{N_A} \frac{1}{\Gamma_{Encoder}} \frac{k_{pm}}{1 + k_{pm}} \Gamma_{Encoder} \frac{N_B}{N_A} k_\phi} \\ &= \frac{\frac{l_r v}{Lh} \frac{v}{l_r} - \frac{N_B^2}{N_A^2} \frac{g}{h} \frac{k_{pm}}{1 + k_{pm}} k_\phi}{1 + \frac{l_r v}{Lh} \frac{v}{l_r} - \frac{N_B^2}{N_A^2} \frac{g}{h} \frac{k_{pm}}{1 + k_{pm}} k_\phi} \end{aligned} \quad (3.23)$$

SYSTEM DEVELOPMENT

The complete transfer function seems to be a PT_1 -block, just like the first model. The above model is the second model designed. The first, which is shown in detail in the appendix A, showed an unsatisfactory relation between the steering and roll angle, as discussed in section 4.

3.5 THE IMPLEMENTATION IN SOFTWARE

On the prototype runs a minimal Debian 10 without any graphic user interface and a 5.4.129 RT kernel. The managing of GPIO is done via device tree overlays and as the `GPIOsysfs` interface is deprecated since Linux 4.8, access to the GPIO from inside a program is handled by `gpiod`, leveraged through the `libgpiod`

Before the central program `bicycle` can be run the pins need to be set up by running the script `pin-config.sh`.

The code, as it is now, represents the basic function necessary for the steering-control. Following the datasheet, [Sen20], the axis were remapped, functions to read Euler angles and gyroscopic measurements were implemented and units can be switched from degrees or $\frac{m}{s}$ to radians or rad/sec. Additionally, the basics for setting up an interrupt are also available.

All functions of the motor and motor controller implemented. The motor can be steered to an steering angle, whose value can be provided in degrees or radians.

The main function consists of a basic do-while-loop, in which the outer feedback angle was implemented.

The programming therefore is far from sophisticated but provides all necessary functions to built a more sophisticated application.

Ideas for further work are discussed in section 4.3.

3.6 MEASURING AND TESTING

A short video snippet on the testing is provided outside this document. It illustrates the current capabilities and limitation of the prototype.

CONCLUSION AND FUTURE WORK

In this work the foundation for a self-balancing bicycle was laid. The prototype has an electric drive and a steering-control was designed and implemented successfully. The accompanying short literature review brought an overview of the current bicycle research and helped to select and form an adequate model for the control.

The literature review showed a variety of models and active research. After grouping the self-balancing approaches by their means of achieving balance, steering control was chosen. A further literature review showed several implementations of steering-control, two of which were tried for the prototype. The first used 3-D printed gears and was abandoned because of unreliability and a combination of metal gears and a toothed belt replaced it.

For the electronics of steering control, a central unit, an inertial measurement unit and a steering motor was chosen. After consideration the BeagleBone Black was chosen as the central single-board computer. For the measurement of position and movement the Bosch BNO055 in form of an Adafruit Breakout was selected. Using its motor characteristics as a guide the EMG49 and its controller board MD49 were picked as the steering drive system.

The software development was done in C and runs on a Linux kernel with `PREEMPT_RT`. Control functions for the motor and the IMU were implemented from scratch. A focus was set on documentation and the comments are structured to allow the use of Doxygen.

The prototype was tested in the field and the results were incorporated.

4.1 MODELING PAIN AND FUTURE MODELS

The literature regarding bicycle modeling is various, engaging and mostly written by authors from the mechanical engineering domain. It is easy to get lost in the diversity of models and theories about self-stability. Additionally, the mathematics required to follow these papers is not part of the standard curriculum for a Bachelor of Science in Computer Science.

Now, one might argue that a simpler model might suffice. In my opinion it does not. Let me illustrate that with an example.

The now used model of the relationship between the steering angle δ and the roll angle ϕ was not the first approach. Before, as is also described in section 2.7, the approach was even simpler. Unfortunately, the old model showed a screwed relationship between the steering and the roll angle. We can intuitively agree, that a bigger steering angle - within limits, of course - leads to a bigger change in the roll angle. But the model did not show that. Only through using the forces which act on and the inertias which rule the inverted pendulum are all

CONCLUSION AND FUTURE WORK

basic movements necessary accounted for, so that the relation between the two angles becomes clearer and not totally counter intuitive and disagreeing with the perused literature.

This more involved model - which still equates to one of the most basic in literature - cost me quite some time. Only a conversation with an mechanical engineer brought an idea of the parallel axis theorem - without it the derivation of the model can not be followed.

Of course, such a model is never finished. The next steps could be the inclusion of the actual kinematic steering angle or even the adaption for a non-constant velocity. Before the model is expanded it needs to be simulated with the actual bike's parameters.

4.2 WOBBLY SCREWS AND UNRELIABLE WIRING

The prototype as it stands now lacks robustness which itself lies in the nature of prototypes. Nevertheless, if the bike is to be an object of teaching and further research the system must be made more robust.

The most critical point are the connections between the components. While the power connections via XT60 are stable enough, the connections between sensors partially depend on simple Dupont wires. This should be replaced by connectors that are more robust.

The drive gear G_A is fixed to the steering motor shaft by a socket screw. This socket screw only has a small contact point with the shaft and is not stable enough. It should be replaced with a bigger ball-ended thrust screw.

The belt was chosen quite long for the possibility of using two motors with one belt. If this is finally decided against, the belt can be replaced with a shorter one which would lessen trouble with belt slack.

4.3 A RT-LINUX WITHOUT ANY RT AND MYSTERIOUS CALIBRATION OUTAGES

The time spent with the model cost time which could have also been spent on programming. Even though the BeagleBone Black and the used custom compiled kernel were chosen for their characteristics concerning time constraints the code, as it is now, uses none of them. It does not even use threading, unfortunately. As it tends to happen, time ran out.

As was mentioned in section 3.3.1, after experimenting with the preemption patch the [PRU-ICSS](#) could be used to generate an exact pulse. This is possible under hard real-time with the [PRU-ICSS](#) and would generate precise timing for comparison of model and implementation.

What is also missing from the code is a simple data collection mechanism. While the software is running the values of the measured and steered angles could be saved to a .csv file to allow for later analysis.

The quite mysterious calibration ritual of the IMU was already mentioned. It could be replaced by a different platform and own sensor fusion could be implemented.

Additionally, a direct motor control would highly benefit the project. The UART communication is a bottleneck and can be quite unreliable. Controlling the DC-motors directly through the BeagleBone Black should be possible and might lead to a big gain in precision.

4.4 FINALLY

Many ideas could not be implemented, but the author hopes that the prototype will nevertheless be used, rebuilt and modified by future students, provide joy - the 50 $\frac{\text{km}}{\text{h}}$ mark has still not been tested - and last, but not least, sharpen their awareness for modes of transport besides the car.

DETAILS OF THE FIRST MODEL

As was mentioned in section 3.4, there was a previous model design. Its details are presented here for documentation and verification purposes.

The goal to balance the bicycle can be modeled as $\phi_{input} = 0$ which describes the desired value of the roll angle to be zero. This is fed into addition with the feedback, which provides the measured roll angle $\phi_{measured}$. The error $e_\phi = \Delta\phi = \phi_{input} - \phi_{measured}$ goes to the first block, the proportional controller.

$$f_1(e_\phi) = k_\phi e_\phi = \gamma_E \quad (A.1)$$

The error is scaled by the proportional coefficient k_ϕ and represents the updated angle γ_E . The angle needs to be expressed in encoder turns, so the motor loop can be entered. This is a linear operation with the constant $\Gamma_{Encoder}$, defined in section 3.2

$$f_2(\gamma_E) = \Gamma_{Encoder} \gamma_E = E_\gamma \quad (A.2)$$

The desired encoder count is the input for the following motor loop. The motor loop possesses its own feedback loop with its proportional controller $P_m = k_{pm} e_m$. The motor loop can be seen isolated in Figure A.1. Since it carries its own feedback it can later be included by using its transfer function. For now, its function is just a placeholder.

$$f_3(E_\gamma) = M(E_\gamma) = E_{motor} \quad (A.3)$$

After the motor loop we transform the encoder turns output E_{motor} by the motor loop into the angle of the steering motor shaft and therefore the driving gear G_A by employing the constant $\Gamma_{Encoder}$ again.

$$f_4(E_{motor}) = \frac{E_{motor}}{\Gamma_{Encoder}} = \gamma_A \quad (A.4)$$

The relation between the angles γ_A, δ is defined by the gear transmission ratio and was established in equation 3.1

$$f_5(\gamma_A) = \gamma_A \frac{N_B}{N_A} = \delta \quad (A.5)$$

Now that we have the steering angle δ we can calculate the lateral distance $\dot{y}_{st}$

$$f_6(\delta) = v \sin \delta = \dot{y} \quad (A.6)$$

To get a distance function y from its velocity derivative $\dot{y}$ we need to integrate

$$f_7(\dot{y}) = \int \dot{y} = y_p \quad (A.7)$$

Not this function can be used for the lateral distance in the pendulum model

$$f_8(y_p) = \sin^{-1} \left(\frac{y}{h} \right) = \phi \quad (A.8)$$

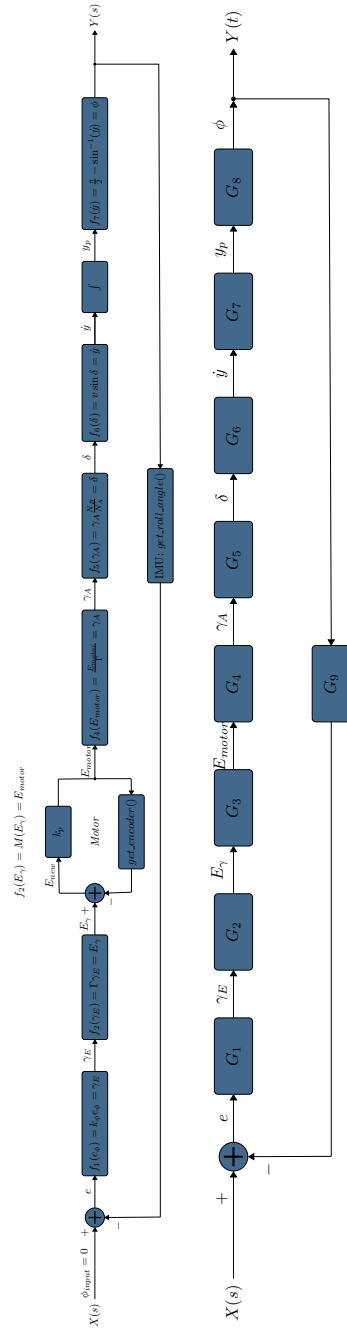


Figure A.1: The Control System Block Diagram of the first modeling attempt

After formulating the single functions in the time domain and orienting them in the block diagram, Figure 3.14, we can continue working towards the frequency domain model. To model the system in the frequency domain above functions need now be transformed via Laplace transform, as described in Section 2.6.

The input as a function of time and later frequency is modeled as

$$\begin{aligned}\phi_{input}(t) &= 0 \\ \phi_{input}(s) &= \mathcal{L}_t\{f_1(t)\} = 0\end{aligned}\tag{A.9}$$

The proportional gain then is modeled as

$$G_1(s) = \frac{\gamma_E(s)}{e(s)} = k_\phi\tag{A.10}$$

To enter the motor loop, the multiplication with the constant $\Gamma_{Encoder}$

$$G_2(s) = \frac{E_\gamma}{\gamma_E(s)} = \Gamma_{Encoder}\tag{A.11}$$

The motor loop carries its own feedback and proportional controller, as can be seen in Figure 3.15. It is therefore modeled as shown in Figure 2.11

$$G_3(s) = \frac{E_{Motor}(s)}{\gamma_E(s)} = \frac{k_{pm}}{1 + k_{pm}}\tag{A.12}$$

Exiting the motor loop requires the constant $\Gamma_{Encoder}$ again

$$G_4(s) = \frac{\gamma_A(s)}{E_{motor}(s)} = \frac{1}{\Gamma_{Encoder}}\tag{A.13}$$

Now for the transmission

$$G_5(s) = \frac{\delta(s)}{\gamma_A} = \frac{N_B}{N_A}\tag{A.14}$$

The next block contains a non-linear function, the sine, and therefore needs to be linearized. The steering angle can be expected to remain small enough as $\delta \leq \frac{\pi}{4}$, since we do not expect steering angles bigger than 45 °. Therefore we can utilize the Taylor polynomial around a operating point $\delta_0 = 0$ which leads to the linearized function and its transfer function

$$\begin{aligned}f_6 \rightarrow \dot{y} &= v \sin(\delta_0) + v \cos(\delta_0)(\delta - \delta_0) \\ &= v \sin(0) + v \cos(0)(\delta - 0) = v\delta \\ G_6(s) &= \frac{\dot{y}(s)}{\delta(s)} = v\end{aligned}\tag{A.15}$$

Now for the integration over $\dot{y}$ to get y

$$G_7(s) = \frac{1}{s} \quad (\text{A.16})$$

The final function has another non-linear function, the arcsine. Using the Taylor polynomial at operating point $y_0 = 0$ leads to the linearized function and its transfer function

$$\begin{aligned} f_8 \rightarrow \phi &= \sin^{-1} \left(\frac{y_0}{h} \right) + \frac{1}{h} \frac{1}{\sqrt{1 - (\frac{y_0}{h})^2}} (y_p - y_0) \\ &= \sin^{-1} \left(\frac{0}{h} \right) + \frac{1}{h} \frac{1}{\sqrt{1 - (\frac{0}{h})^2}} (y_p - 0) \\ &= \frac{y_p}{h} \\ G_8(s) &= \frac{\phi(s)}{y_p(s)} = \frac{1}{h} \end{aligned} \quad (\text{A.17})$$

$$\begin{aligned} G_{all} &= \frac{G_8 G_7 G_6 G_5 G_4 G_3 G_2 G_1}{1 + G_9 G_8 G_7 G_6 G_5 G_4 G_3 G_2 G_1} \\ &= \frac{\frac{1}{h} \frac{1}{s} v \frac{N_B}{N_A} \frac{1}{\Gamma_{Encoder}} \frac{k_{pm}}{1+k_{pm}} \Gamma_{Encoder} k_\phi}{1 + \frac{1}{h} \frac{1}{s} v \frac{N_B}{N_A} \frac{1}{\Gamma_{Encoder}} \frac{k_{pm}}{1+k_{pm}} \Gamma_{Encoder} k_\phi} \\ &= \frac{k_{phi} k_{pm} N_B v}{k_\phi k_{pm} N_B v + (1 + k_p) N_A h s} \end{aligned} \quad (\text{A.18})$$

The complete transfer function seems to be a PT_1 -block, like a low pass of the first order, of the general form

$$G_{PT_{make1}}(s) = \frac{Y(s)}{U(s)} = \frac{K}{1 + Ts} \quad (\text{A.19})$$

with the gain factor K and the time constant T of

$$\begin{aligned} K &= 1 \\ T &= \frac{(1 + k_p) N_A h}{k_{phi} k_{pm} N_B v} \end{aligned} \quad (\text{A.20})$$

DETAILS OF THE SOFTWARE IMPLEMENTATION

The transmission ratio is provided via global constants. The variables `encoder_to_angle=A` and `steering_angle_to_encoder_ratio=B` depend on the gear transmission ratio i , seen in equation 3.1, the constant $\Gamma_{Encoder}$ from equation 3.7 and the encoder count E_{motor} from section 3.3.3. They are the inverse of each other:

$$\begin{aligned} A &= \frac{E_{Motor}i}{2\pi} \\ B &= \frac{2\pi}{E_{Motor}} \end{aligned} \tag{B.1}$$

comms are set up

WIRING DIAGRAM

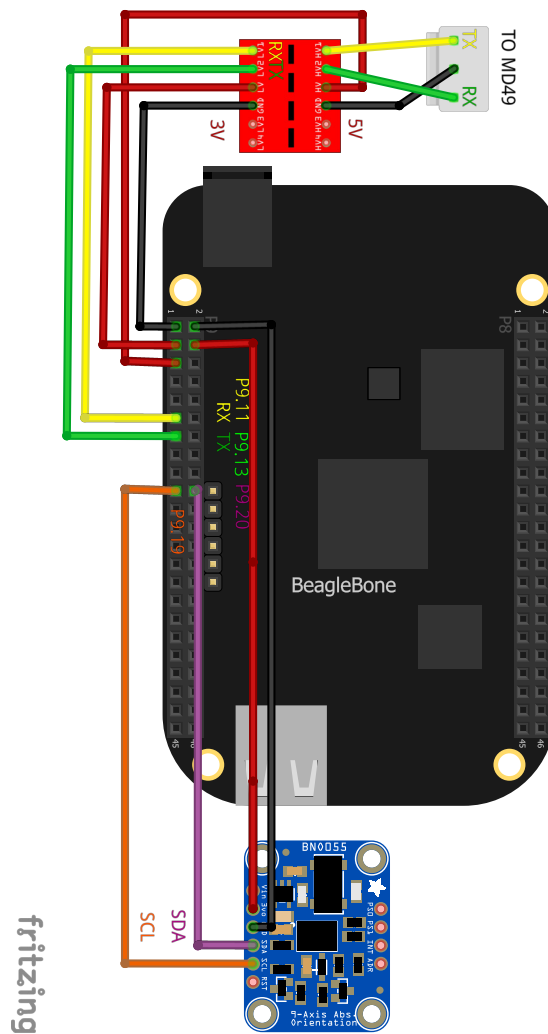


Figure C.1: Wiring Diagram generated with fritzing[fri21]. The fritzing file is included in the accompanying source folder.

LIST OF PARTS AND USED CONNECTOR TYPES

D

- The driving motor:
 - 48 V, 1500 W 26 inch E-Bike conversion from Shanghai CSC Sports Co. Ltd[Ltd21b]
 - used connector types:
 - * XT60 to power supply
- The steering motor:
 - EMG49 - Gear motor with encoder [Lim21a] from Devantech[Ltd21a]
 - MD49 - 24V 5A dual H-bridge motor driver[Lim21b] from Devantech[Ltd21a]
 - Needed connector type:
 - * MD49:
 - 3-pin JST-PHR-3 to BeagleBone Black 3.96 mm Excon type 3961 pitch terminal to motor power
 - 4-pin JST-PHR-4 for motor hall sensor
 - * EMG49:
 - 3.96 mm Excon type 3961 pitch terminal for power
 - 4-pin JST-PHR-4 for hall sensor
- The power supply:
 - 2 24 V BIB-0240100LT Lithium-Ionen-Batteries from Lion-Tec[Lio21]
 - RSD-30G-5 DC-to-DC converter [Mea21]
 - Schneider Electric XALK178F Emergency Stop[Ele21]
 - Berker 3966 Pull Switch[Ber21]
 - used connector types:
 - * Lithium-Ionen-Batteries
 - two blade contact from manufacturer to battery
 - XT60
 - * Rest:
 - screw terminals
- The steering control:
 - BeagleBone Black Rev. C [Fou20]
 - Adafruit BNO055 Breakout[Ind21]
 - SparkFun Logic Level Converter 12009[Spa21]
 - Needed connector type:

- * BeagleBone Black:
 - 2 46-pin 0.1 inch headers
 - 3.5 mm 5V DC barrel jack for power supply

- * Rest:
 - 2.54 mm header

- The belt transmission:

- 30-teeth gear [[Mäd21d](#)]
- 44-teeth gear [[Mäd21e](#)]
- Toothed belt [[Mäd21b](#)]
- taper [[Mäd21c](#)]
- socket screw [[Mäd21a](#)]

ERKLÄRUNG ZUR ABSCHLUSSARBEIT

Bitte dieses Formular zusammen mit der Abschlussarbeit abgeben!

Erklärung zur Abschlussarbeit

gemäß § 25, Abs. 11 der Ordnung für den Bachelorstudiengang Informatik
vom 06. Dezember 2010:

Hiermit erkläre ich ~~Herr~~ / Frau

Anna Ranz

Die vorliegende Arbeit habe ich selbstständig und ohne Benutzung
anderer als der angegebenen Quellen und Hilfsmittel verfasst.

Frankfurt am Main, den

15.8.21



Unterschrift der Studentin / des Studenten

GLOSSARY

Coriolis force is a physical force that acts on a body in motion inside a referential frame which rotates with respect to an inertial frame.. [11](#)

Gerber is a vector format for printed circuit board designs. [27](#)

Lorentz force is a combination of electric and magnetic force on a point charge by a electromagnetic field. [11](#)

pitch denotes the bike's motion about the pitch axis, which lies along the y -axis in this text. [5](#)

PREEMPT_RT is the name of a Linux kernel patch that enables real-time preemption. [27](#), [35](#)

roll denotes the bike's motion about the roll axis, which lies along the x -axis in this text. [5](#)

yaw denotes the bike's motion about the yaw axis, which lies along the z -axis in this text. [5](#)

ACRONYMS

I²C Inter-Integrated Circuit. [13, 14](#)

9-DOF nine degrees of freedom. [27](#)

BOM Bill of Materials. [27](#)

CMG Control Moment Gyroscope. [9, 10](#)

CPU Central Processing Unit. [27](#)

DC Direct Current. [11](#)

FDM Fused Deposition Modeling. [24](#)

GPIO general-purpose input/output. [27, 34](#)

IMU Inertial Measurement Unit. [10, 11, 13, 26, 27, 35, 37](#)

MEMS Micro-Electro-Mechanical Systems. [11](#)

PETG Polyethylene terephthalate glycol, a thermoplastic polyester. [24](#)

PID Proportional–Integral–Derivative. [10](#)

PLA Polylactic acid, a thermoplastic polyester. [24](#)

PRU-ICSS Programmable Real-Time Unit Subsystem and Industrial Communication Subsystem. [27, 36](#)

RT Real-Time. [27, 34](#)

RX Receive. [14](#)

SCL Serial Clock Line. [14](#)

SDA Serial Data Line. [14](#)

SoC System on Chip. [27](#)

TX Transmit. [14](#)

UART Universal Asynchronous Receiver-Transmitter. [13, 14, 37](#)

BIBLIOGRAPHY

-
- [ÅKL05] K.J. Åström, R.E. Klein, and A. Lennartson. “Bicycle dynamics and control: adapted bicycles for education and research”. In: *IEEE Control Systems* 25.4 (Aug. 2005), pp. 26–47. URL: <https://ieeexplore.ieee.org/document/1499389/> (visited on 06/01/2021).
- [ÅM21] K. J. Åström and R. M. Murray. *Feedback systems: an introduction for scientists and engineers*. Second edition. Princeton, USA: Princeton University Press, 2021.
- [Ber21] Berker. 3966 datasheet. 2021. URL: https://www.hager.de/files/download/0/15761_1/0/Berker_Produktdatenblatt_3966.pdf (visited on 07/07/2021).
- [BKB19] K. Berns, B. Köpper, and Schürmann B. *Technische Grundlagen Eingebetteter Systeme*. 1st ed. Wiesbaden, Germany: Springer Vieweg, 2019.
- [Bul17] V. Bulsink. “Bicycling stability : simulations & experiments to improve cycling safety for older cyclists”. PhD. Enschede, The Netherlands: University of Twente, Dec. 2017. URL: <http://purl.org/utwente/doi/10.3990/1.9789036544436> (visited on 01/05/2021).
- [Car99] E. Carvallo. *Théorie Du Mouvement Du Monocycle et de La Bicyclette*. Paris, France: Gauthier- Villars, 1899.
- [Co21] Murata Manufacturing Co. *Mutata Boy*. 2021. URL: https://corporate.murata.com/en-eu/more_murata/robots/mboy (visited on 07/07/2021).
- [Del19] TU Delft. *Smart motor in handlebars prevents bicycles from falling over*. 2019. URL: <https://www.tudelft.nl/en/2019/tu-delft/smart-motor-in-handlebars-prevents-bicycles-from-falling-over/> (visited on 07/07/2021).
- [Del21] TU Delft. *Bicycle Dynamics*. 2021. URL: <http://bicycle.tudelft.nl/> (visited on 07/07/2021).
- [Den+18] W. Deng et al. “Towards Automated Bicycles: Achieving Self-Balance Using Steering Control”. In: *Proc. of the ASME 2018 Dynamic Systems and Control Conference* (Atlanta, Georgia, USA). American Society of Mechanical Engineers, Sept. 2018. URL: <https://doi.org/10.1115/DSCC2018-9244> (visited on 01/05/2021).
- [Döh55] E. Döhning. “Die Stabilität von Einspurfahrzeugen”. In: *Forschung im Ingenieurwesen* 21 (Mar. 1955), pp. 50–62.
- [Ele21] Schneider Electric. *XALK178F datasheet*. 2021. URL: <https://www.se.com/ww/en/product/download-pdf/XALK178F> (visited on 07/07/2021).

BIBLIOGRAPHY

- [Faw19] Z. Fawaz. *Design and Control of a Self-balancing Bicycle Using an Electric Linear Actuator*. Dearborn, USA, 2019. URL: <https://deepblue.lib.umich.edu/handle/2027.42/148871> (visited on 05/09/2021).
- [Fed18] European Cyclists' Federation. *The benefits of cycling - Unlocking their potential for Europe*. 2018. URL: https://ecf.com/sites/ecf.com/files/TheBenefitsOfCycling_final-v2.pdf (visited on 07/07/2021).
- [Fou20] BeagleBoard.org Foundation. *BeagleBone Black System Reference Manual*. 2020. URL: <https://github.com/beagleboard/beaglebone-black/wiki/System-Reference-Manual> (visited on 01/05/2021).
- [Fou21a] BeagleBoard.org Foundation. *BeagleBone Black Repository*. 2021. URL: <https://github.com/beagleboard/BeagleBone-Black> (visited on 07/07/2021).
- [Fou21b] Raspberry Pi Foundation. *Raspberry Pi 4 Tech Specs*. 2021. URL: <https://www.raspberrypi.org/products/raspberry-pi-4-model-b/specifications/> (visited on 07/07/2021).
- [fri21] fritzing. *fritzing Webpage*. 2021. URL: <https://fritzing.org/> (visited on 07/07/2021).
- [GM95] N.H. Getz and J.E. Marsden. "Control for an autonomous bicycle". en. In: *Proceedings of 1995 IEEE International Conference on Robotics and Automation*. Vol. 2. Nagoya, Japan: IEEE, 1995, pp. 1397–1402. URL: <http://ieeexplore.ieee.org/document/525473/> (visited on 01/05/2021).
- [HZS15] J. He, M. Zhao, and S. Stasinopoulos. "Constant-velocity steering control design for unmanned bicycles". In: *2015 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. Zhuhai: IEEE, Dec. 2015, pp. 428–433. URL: <http://ieeexplore.ieee.org/document/7418805/> (visited on 01/05/2021).
- [Ind21] Adafruit Industries. *Adafruit 9-DOF Absolute Orientation IMU Fusion Breakout - BNO055*. 2021. URL: <https://www.adafruit.com/product/2472> (visited on 01/05/2021).
- [Ins21] Texas Instruments. *AM3358 Product Page*. 2021. URL: <https://www.ti.com/product/AM3358> (visited on 07/07/2021).
- [Isa07] W. Isaacson. *Einstein: his life and universe*. New York, USA: Simon & Schuster, 2007.
- [Koo+11] J. D. G. Kooijman et al. "A Bicycle Can Be Self-Stable Without Gyroscopic or Caster Effects". In: *Science* 332.6027 (Apr. 2011), pp. 339–342. URL: <https://www.sciencemag.org/lookup/doi/10.1126/science.1201959> (visited on 01/05/2021).
- [KS10] F. Klein and A. Sommerfeld. *Über die Theorie des Kreisels*. Leipzig, Germany: Teubner, 1910.

- [Lim06] R. S. Limebeer D. J. nd Sharp. "Bicycles, Motorcycles, and Models". In: *IEEE Control Systems* 26.October 2006 (Sept. 2006), pp. 34–61. URL: <https://ieeexplore.ieee.org/document/1700044> (visited on 02/01/2021).
- [Lim21a] Devantech Limited. *EMG49, mounting bracket and wheel specification*. 2021. URL: <https://www.robot-electronics.co.uk/htm/emg49.htm> (visited on 07/07/2021).
- [Lim21b] Devantech Limited. *MD49 - Dual 24 Volt 5 Amp H Bridge Motor Drive*. 2021. URL: <https://www.robot-electronics.co.uk/htm/md49tech.htm> (visited on 07/07/2021).
- [Lio21] LionTec. *Datenblatt Lithium-Ionen-Batterie BIB-0240100LT*. 2021. URL: https://www.liontecshop.de/shop/handle_downloads.php?context=downloads&id=1162&file=Datenblatt%20BIB-0240100LT.pdf (visited on 07/07/2021).
- [Ltd21a] Devantech Ltd. *Industrial Electronic Systems*. 2021. URL: <https://devantech.co.uk/> (visited on 07/07/2021).
- [Ltd21b] Shanghai CSC Sports Co. Ltd. *48V E-Bike Conversion Kit*. 2021. URL: <https://www.carbonspeedcycle.com/48v-motor-wheel-with-tire.html> (visited on 07/07/2021).
- [Mäd21a] Mädler. *Ball-Ended Thrust Screws*. 2021. URL: <https://www.maedler.de/product/1643/3/kugeldruckschrauben-stahl> (visited on 07/07/2021).
- [Mäd21b] Mädler. *Polyurethane timing belt Art-Nr.16285900*. 2021. URL: <https://www.maedler.de/product/1643/1616/955/zahnriemen-profil-t-5-breite-25-mm> (visited on 07/07/2021).
- [Mäd21c] Mädler. *Taper bush Art.-Nr.6225032540*. 2021. URL: <https://www.maedler.de/product/1643/1615/362/1656/448/1983/taper-spannbuchsen-mit-zoelliger-bohrung> (visited on 07/07/2021).
- [Mäd21d] Mädler. *Timing belt pulley 30T Art.-Nr.16243000*. 2021. URL: <https://www.maedler.de/product/1643/1616/986/zahnriemenraeder-t5-fuer-riemenbreite-25-mm> (visited on 07/07/2021).
- [Mäd21e] Mädler. *Timing belt pulley 44T Art.-Nr.16277444*. 2021. URL: <https://www.maedler.de/product/1643/1616/986/zahnriemenraeder-t5-taper-fuer-riemenbreite-25-mm> (visited on 07/07/2021).
- [Mea21] Meanwell. *RSD-30 series datasheet*. 2021. URL: <https://www.meanwell.com/Upload/PDF/RSD-30/RSD-30-SPEC.PDF> (visited on 07/07/2021).
- [Mei+07] J.P. Meijs et al. "Linearized dynamics equations for the balance and steer of a bicycle: a benchmark and review". In: *Proceedings of the Royal Society A* 463 (2084 Aug. 2007), pp. 1955–1982.

BIBLIOGRAPHY

- [Moo12] J. K. Moore. “Human Control of a Bicycle”. Doctoral Dissertation. Enschede, The Netherlands: University of California, Davis, Aug. 2012. URL: <https://moorepants.github.io/dissertation/index.html> (visited on 01/05/2021).
- [Sen20] Bosch Sensortec. *BNO055 data sheet: Intelligent 9-axis absolute orientation sensor*. 2020. URL: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bno055-ds000.pdf> (visited on 01/05/2021).
- [SHB18] D. Schramm, M. Hiller, and R. Bardini. “Einspurmodelle”. de. In: *Modellbildung und Simulation der Dynamik von Kraftfahrzeugen*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2018, pp. 225–254. DOI: 10.1007/978-3-662-54481-5_10. URL: http://link.springer.com/10.1007/978-3-662-54481-5_10 (visited on 07/23/2021).
- [Spa21] Sparkfun. *SparkFun Logic Level Converter - Bi-Directional*. 2021. URL: <https://www.sparkfun.com/products/12009> (visited on 07/07/2021).
- [TM04] Y. Tanaka and T. Murakami. “Self Sustaining Bicycle Robot with steering controller”. In: *The 8th IEEE International Workshop on Advanced Motion Control, 2004. AMC '04*. Kawasaki, Japan: IEEE, 2004, pp. 193–197. ISBN: 978-0-7803-8300-5. URL: <http://ieeexplore.ieee.org/document/1297665/> (visited on 01/05/2021).
- [Uni21] Cornell University. *Biorobotics and Locomotion Lab*. 2021. URL: http://ruina.tam.cornell.edu/research/topics/bicycle_mechanics/overview.html/ (visited on 07/07/2021).
- [Uta+05] A. Utano et al. “Automatic control of bicycles with a balancer”. In: *Proceedings, 2005 IEEE/ASME International Conference on Advanced Intelligent Mechatronics*. Monterey, CA: IEEE, 2005, pp. 1245–1250. URL: <http://ieeexplore.ieee.org/document/1511181/> (visited on 05/09/2021).
- [Whi19] Francis J. W. Whipple. “The Stability of the Motion of a Bicycle”. In: *Quarterly Journal of Pure and Applied Mathematics* 30 (2019), pp. 312–348.
- [WSP20] D. G. Wilson, T. Schmidt, and J. M. Papadopoulos. *Bicycling Science*. 4th ed. Cambridge, USA: MIT Press, 2020.
- [Yet+14] H. Yetkin et al. “Gyroscopic stabilization of an unmanned bicycle”. In: *2014 American Control Conference*. Portland, USA: IEEE, June 2014, pp. 4549–4554. URL: <http://ieeexplore.ieee.org/document/6859392/> (visited on 05/09/2021).